Sreeraaghav Raja
4/26/26
40990068

EE Design 1 Technical Report
Final Project
Luminous Library

Introduction

Luminous Library was the brainchild of an individual who was driven to do an IoT project but did not have the time to properly implement the IoT part of the project. The purpose of this project was to embrace my passion for reading and solve an issue that I've had in the past: logging my reading progress in a gamified way. This project serves as a high-fidelity prototype of a book logger that integrates both analog inputs and outputs (DACs, potentiometers, Photoresistors) and digital inputs and outputs (switches, buttons, LEDs) to implement a functional book logger design. The prototype implements visual and auditory feedback for the logging of a book upon a completion stage. There are also 2 additional modes where one recommends a book for each of the 16 basic genres and another that adds 16 new genres to log.

Methods

Background Information

The necessary information required for this project lies more in software than in hardware. Nevertheless, I will first explain the analog input sensors in depth before I explain anything related to the software. The project also utilized an array of digital inputs and outputs, but they were much simpler as they were switches, buttons, and LEDs that did not use Pulse-Width Modulation (PWM) therefore they will be briefly explained after the analog components.

Analog Input Sensor: Potentiometer

The two analog input sensors utilized in this project are the potentiometer and the photoresistor. The potentiometer is an incredibly unique analog component because of the way it incorporates mechanical engineering elements into its design. The potentiometer uses a small resistive strip and a wiper attached to knob to control the internal resistance of the component.

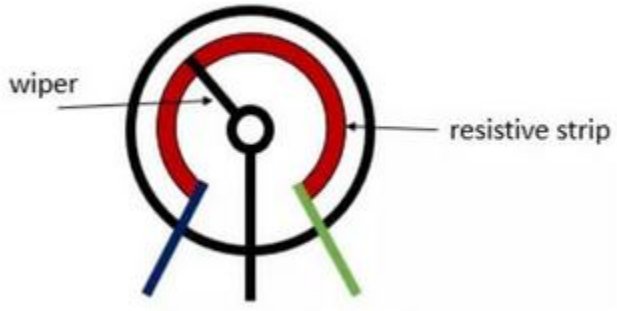

Figure 1: Potentiometer Internal Design

The physics involved in this component are directly related to the fundamental theory behind resistance [1]. Resistance is a measure of resistivity over a ratio between length and area. Thus, the equation is:

$$R = \rho\, {}^{l}/_{A}$$

Where ρ is resistivity, a measure of how much a component resists electrical current, and L and A correspond to the length of the strip and the surface area of the cross section where the current is flowing. Therefore, as the length increases while cross-sectional area and resistivity, an intrinsic property of the material, remain constant the resistance increases.

Analog Input Sensor: Photoresistor

The potentiometer is basic in terms of physics complexity in comparison to a photoresistor. The photoresistor is a marvel of solid-state device physics as it incorporates an intuitive implementation of pseudo-diode. A diode operates by using a voltage buildup to enable current to flow across a junction between the P-Type and N-Type semiconductors. However, the photoresistor accomplishes this with photons, making it a direct semiconductor. Additionally, photoresistors are also classified based on the type of material they are comprised of. There are intrinsic photoresistors which have "undoped materials like silicon and germanium" and extrinsic photoresistors which require dopants and are more susceptible to other wavelengths of light [2]. Figure 2 emphasizes why I used a Cadmium Sulfide (CdS) cell as it is sensitive over full range of visible light which is the easiest to test [2].

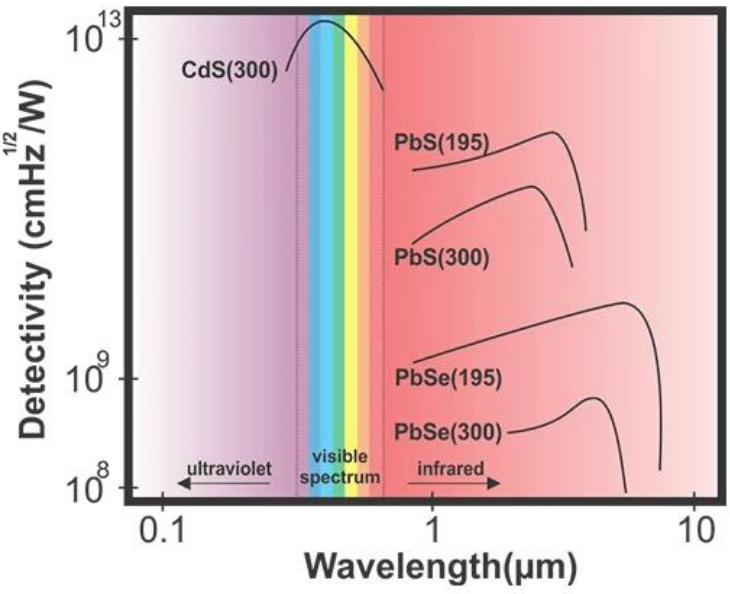

Figure 2: Photoresistor Material Operation Ranges

The physical properties of a photoresistor that drives its operation are linked to semiconductor properties. Semiconductors require a stimulus of some sort which increases the number of free electrons which bends the conduction band and valence band to allow for a smoother transfer of free electrons from the n-type region to the p-type region [3]. However, a

photoresistor only utilizes half of this since it is merely a single material such as a CdS cell. Instead of requiring free electrons to bridge the gap between semiconductors, the light increases the number of free electrons in the conduction band, making the flow of current much easier and the voltage across the resistor drops [3].

The equation for this circuit is simply a basic voltage divider. The CdS cell that I used was rated to be 10kΩ for a fully dark condition, so upon an influx of light the resistance drops and the voltage. The best resistor to use for this scenario is a 10kΩ resistor so that the voltage drops to 1.67V on a fully dark condition and 5V for a fully light condition. If the fixed resistor value is connected after the photoresistor to ground. This is apparent in the schematic figure used later, but the equation for this circuit would be:

$$V_{out} = \frac{R_{fixed}}{R_{photoresistor} + R_{fixed}}$$

Analog Output: DAC

The explanation for the DAC will remain brief as it was a fundamental component of this lab but was also explained in previous modules. The Digital-to-Analog Convertor (DAC) utilizes sampling of a signal to recreate / reconstruct an analog signal from digital values. Thus, it converts from binary values to estimated analog values based on the type of converter. The number of bits used to detect the smallest change in voltage is called the resolution of the DAC. As a result, the 12-bit DAC, the LTC1661 IC, can output 1mV of difference for a change to the LSB of the input signal. Therefore, this DAC is accurate enough for its purpose as a tone to verify the proper logging of a book [4].

The DAC also interfaces with the Raspberry Pi Pico 2W using the Serial-Peripheral Interface (SPI) communication protocol. This protocol is fast because it is synchronous, meaning it is clocked, and it has 4 wires of communication, making it fully duplex. This means that SPI is the perfect protocol to interact with DAC as its speed resolves any bottleneck with the timing of a signal conversion. SPI often has 4 different modes based on the idle state of the clock and the edge that it is sampled on. The LTC1661 uses SPI Mode 0 which was deduced through the documentation [5]. This was explored in a previous lab and most of the information regarding the implementation of the DAC is not vital in terms of the larger implementation of this project.

Digital Input: Switches and Buttons

The explanation of switches and buttons are trivial to explain as they are some of the most fundamental circuit components. Effectively, the switches are all a pull-up configuration meaning that they are VCC, approximately 3.3V for my implementation, when the switch is open and 0V when the switch is closed. This saves power for the rest of the circuit as the components are all connected to a common ground rail. However, the extra resistors could have been avoided since the Raspberry Pi Pico 2W has internally configured pull-up circuitry for all the GPIO pins. Nevertheless, the most common diagram for a pull-up circuit is shown in Figure 3 [6].

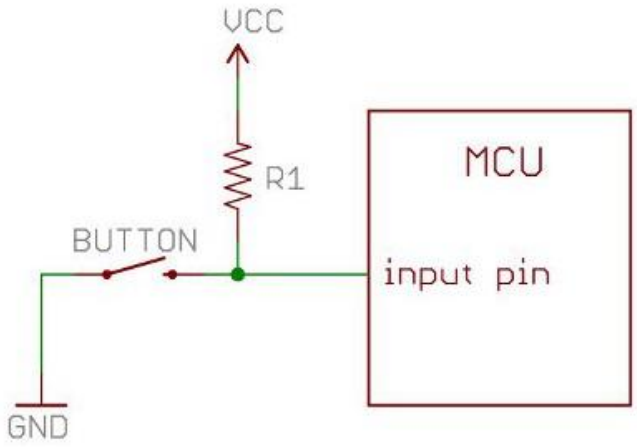

Figure 3: Pull-Up Circuit Diagram

Digital Output: LEDs

I used 5 red LEDs for this project to display the progress when reading a book and log that into the system. LEDs require a voltage difference across their terminals that would overcome the threshold voltage of the depletion region of the semiconductor. The most common threshold voltage for the red LED is between 1.6V to 2.0V. Interestingly, the threshold voltage is used interchangeably with forward voltage even though one is an intrinsic property of the material and the other is the voltage placed across the terminals of the semiconductor that will allow current to flow freely. Nevertheless, they have the same meaning in this case since the threshold voltage of a Red LED is at minimum 1.6V. There was an issue with two LEDs

LED COLORS AND MATERIALS

Color	Wavelength Range (nm)	Forward Voltage (V)	Material
Ultraviolet	< 400	3.1 - 4.4	Aluminium nitride (AlN) Aluminium gallium nitride (AlGaN) Aluminium gallium indium nitride (AlGaInN)
Violet	400 - 450	2.8 - 4.0	Indium gallium nitride (InGaN)
Blue	450 - 500	2.5 - 3.7	Indium gallium nitride (InGaN) Silicon carbide (SiC)
Green	500 - 570	1.9 - 4.0	Gallium phosphide (GaP) Aluminium gallium indium phosphide (AlGaInP) Aluminium gallium phosphide (AlGaP)
Yellow	570 - 590	2.1 - 2.2	Gallium arsenide phosphide (GaAsP) Aluminium gallium indium phosphide (AlGaInP) Gallium phosphide (GaP)
Orange / Amber	590 - 610	2.0 - 2.1	Gallium arsenide phosphide (GaAsP) Aluminium gallium indium phosphide (AlGaUInP) Gallium phosphide (GaP)
Red	610 - 760	1.6 - 2.0	Aluminium gallium arsenide (AlGaAs) Gallium arsenide phosphide (GaAsP) Aluminium gallium indium phosphide (AlGaInP) Gallium phosphide (GaP)
Infrared	> 760	> 1.9	Gallium arsenide (GaAs) Aluminium gallium arsenide (AlGaAs)

Figure 4: Forward Voltages for LEDs by Color

when soldering to the PCB which will be explained in a later section, but that may have been caused by the Pico not supplying at least 1.6V to those two LEDs.

Software: FreeRTOS

The most interesting part of this project is the implementation of FreeRTOS for this project. The reason why I used a Real-Time Operating System (RTOS) for this project was to practice the skills that I acquired from Microprocessor Applications II, an extremely useful class that I have suffered through in my time at university. There are many issues with using an RTOS specifically the complexity, but the advantage of lower power consumption was worthwhile for a project like this. Moreover, the RTOS was implemented because it was challenging and would increase my interest in my project.

A RTOS effectively transitions between different **tasks/threads** based on a scheduler. While a microprocessor can only execute one task at a time unless there are multiple cores, an RTOS switches between different tasks fast enough to make it appear as though they are all happening concurrently. This was necessary for my project because of one specific part of the project: the progress bar. The progress bar worked by reading the potentiometer value and converting it into a digital value and lighting up a certain number of LEDs based on the voltage. I also needed it to appear on the LCD at the same time [8]. Therefore, the easiest way to accomplish this was to use a RTOS. The easiest way to schedule tasks is to use a Round Robin Priority Scheduler which would give equal time to each task when there are no delays, but increased delays would cause certain tasks to have higher priority than other tasks. For instance, one would assign a thread that handles the chime as higher priority than any other thread because it requires significantly more resources than other functions. Moreover, threads could also be given higher priority than other threads which could lead to starvation of a task. That was not a big issue for my project as the state logic helped handle any discrepancies in CPU time between tasks.

The final concepts related to a RTOS that were significant for this project were semaphores and mutexes. Semaphores and mutexes are effectively “locks” that are used to prevent other threads from using a certain resource while it is used by another thread. The difference between the two is trivial for this implementation but effectively mutexes are more restrictive on which task can unlock and lock then whereas semaphores are more free flowing with that restriction. Moreover, a semaphore implements a “yield” functionality which will prevent a thread from running when a resource is being used but queue it up to start once that resource is no longer being used [9]. Overall, this software implementation was a lot more complicated to explain than it was to implement.

Cyber-Informed Engineering Best Practices

Advantage 1: Secure Information Architecture

The advantage of my project is that there are limited consequences to it being attacked in any capacity. For instance, the information is locally stored in an array in the system's memory, and the information is not critical information of an individual either.

Advantage 2: Engineering Controls

The second advantage of my system is that it is entirely deterministic, meaning that it is completely controlled by digital and analog inputs. Therefore, there is no possibility of randomness in the design, protecting it from hackers or other avenues of attack. I even opted to manually use pull-up resistors instead of programming them to prevent another avenue of attack through software.

Advantage 3: Design Simplification

I made my program software heavy, but the software does not have any non-critical functionality that could be easily attacked. There is an excess of features, but by having the system only access these features through deterministic controls, it simplified the software implementation. Thus, the software is easier to debug in the event there is an attack. Moreover, the learning curve of the RTOS also makes it harder to attack in any case.

Advantage 4: Layered Defenses

The project must be directly accessed to be hacked into or else the Pico 2W must be reprogrammed to manipulate its functionality. The program is stored in nonvolatile memory as it was loaded into the Pico using a UF2 file. Therefore, that is one layer of defense for this project. Another layer of defense is the lack of wireless communication for this project. This prevents another avenue of attack, making this design extremely safe. However, a disadvantage is that the PCB had to be externally produced, allowing there to be an avenue to manipulate the design. If I had etched my own PCB, that would have removed this avenue of attack.

Advantage 5: Interdependency Evaluation

The only interdependency in this system is that the power module requires there to be a battery system. The system was not designed for there to be any other form of power supply input in mind initially. The design was tested with a current limiting source, but that was later avoided in favor of a simpler battery system. While this system has reduced instability the possibility of instability in this system would be caused by dysfunctional battery caps of some which can just be desoldered.

Design Steps:

1. Create a schematic of design and choose components that simplify the design (e.g. use a DIP switch pack instead of 5 individual switches)
2. Breadboard each individual segment and program them in the following order:
    a. ADC with Potentiometer
    b. Switches
    c. LEDs for Progress Bar
    d. Button
    e. Display
    f. DAC
3. Implement the entire RTOS functionality after testing each component
4. Design Altium Schematic and PCB after ensuring the circuit functioned properly
5. Solder and Test PCB after rigorous testing on breadboard

Software Block Diagram

The complexity of my code necessitates that I simplify the software block diagram for the sake of comprehensibility. A RTOS is not that complex it implements once a person understands the basics, but the sheer volume of code makes it unnecessary to explain each line in this software block diagram. Therefore, I will only explain the code in Tasks.c as that file integrates the content from the rest of the files in the project. Moreover, I will not go line by line for Tasks.c as that would be too verbose and will focus on the basic functionality of each thread instead.

Progress Task:

This program starts with the progress task which just converts the voltage value from the potentiometer to a digital value and converts that into a percentage. It is also used to convert turn on a certain number of LEDs that correspond to percentage. It is important to note that the display only changes when the progress value changes by a significant amount. Once the progress value is registered, the display shows “Registration Complete.”

Button Task:

The button task also has a corresponding Button Interrupt Service Routine (ISR) that signals a semaphore and enters the button task. The Button Task waits for that button to be pressed and signals the start of different modes based on previously enabled flags. If the start flag isn’t enabled, then that indicates the start of a new registration, so the flag is set true. The book recommendation flag is set if the photo mode is DARK. Otherwise, if the book rec flag is set and the start flag is set, then the next button press exits that mode. Otherwise, there are other modes as well. For instance, if the genre flag and progress flag aren’t set, then the genre flag is set on the next button press. If the genre flag is set and the progress flag isn’t, then the

progress flag is set on the next button press. Finally, if both flags are set, then the progress and genre are logged into an array.

Switch Task:

The switch task just reads the values from each of the switches and condenses them into a 4-bit value. That value is used differently based on what mode the program is in. If the mode is Book Rec, then the switch value is used to choose a genre to provide a recommendation. This is the same for the genre select mode, both the DARK and BRIGHT mode, but that is selected based on a ternary operator in the Switch Task.

Chime Task:

This task is the shortest task as it just waits for the chime semaphore to be signaled before the chime starts. The program waits for the start flag and log mode flags to be enabled before the chime ends.

Photo Task:

This task is also relatively short since it reads the voltage from the photoresistor, converts that to a digital value, and chooses the mode based on what the converted voltage value is. If the voltage is greater than or equal to 2.0V, then the mode is DARK, if the voltage is less than 0.5V then the mode is BRIGHT, otherwise it is NORMAL mode.

Log Task:

This task is meant to print the previously registered book genres and their progress. The program reads the switch value and waits for it to be debounced before stopping the chime and setting the log mode flag. If there is nothing logged yet, the log index = 0, so the program prints “No Logs Found” otherwise the program prints the genre and progress to the LCD every 1.5 seconds. Finally, when the switch is released, the display clears and the default screen with the project name is printed on the LCD display.

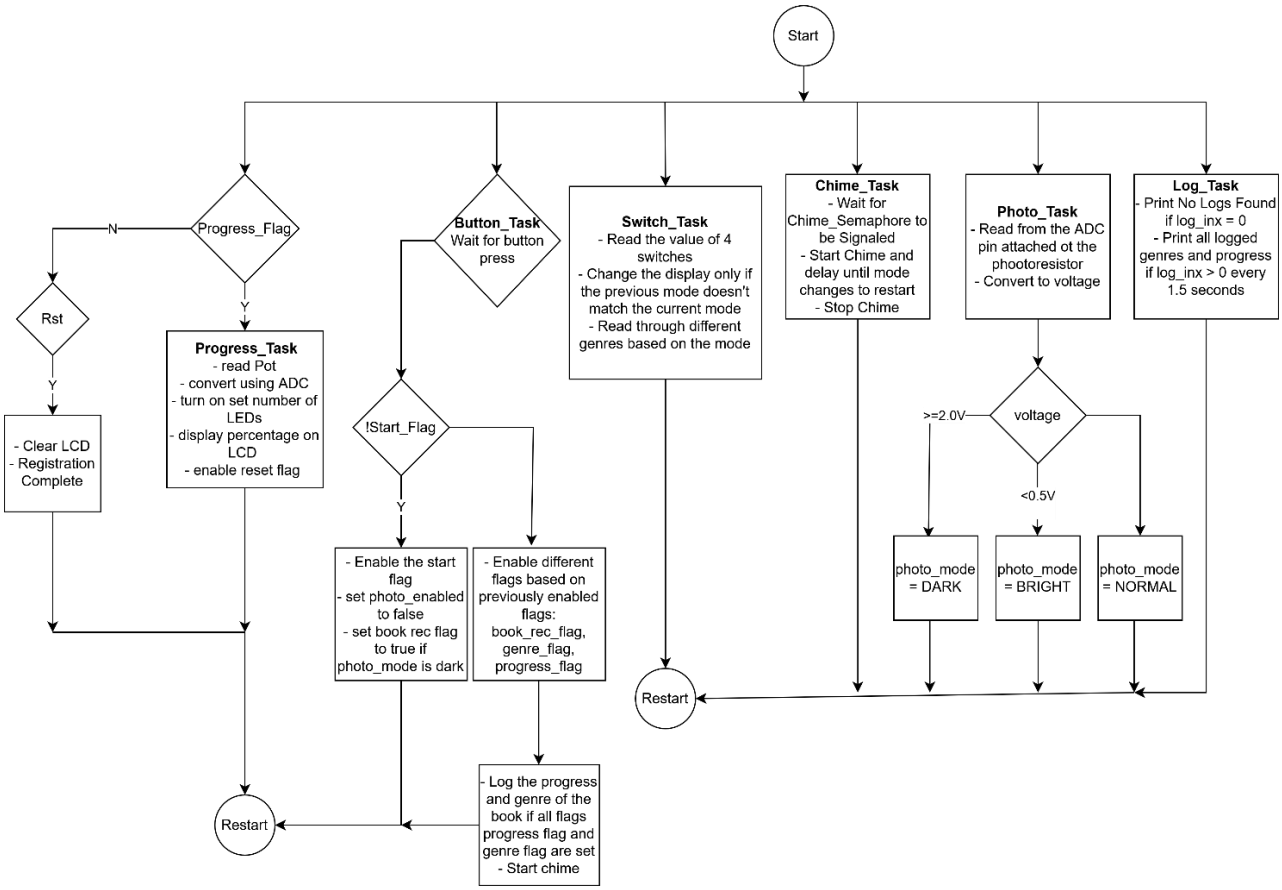

Figure 5: Software Bock Diagram

Hardware Block Diagram

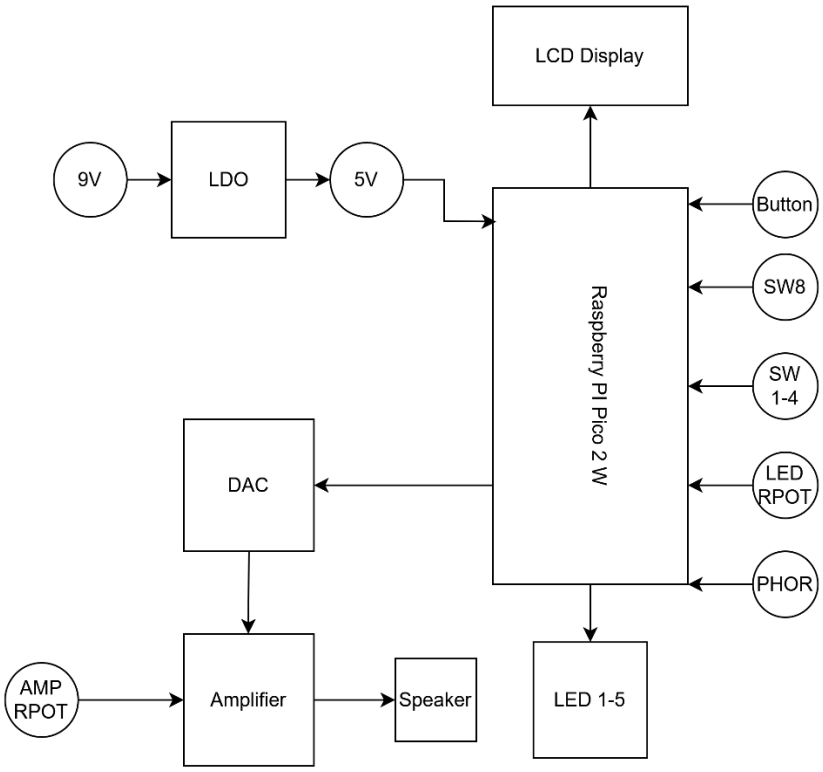

Figure 6: Hardware Block Diagram

This hardware block diagram is simple and matches the schematic. The Raspberry Pi Pico 2 and the Raspberry Pi Pico 2 have the same footprint so it would not matter which one I used since I didn't implement wireless functionality for my project. Moreover, there are 6 different digital inputs: 5 switches and 1 button. There are 2 analog inputs, the LED potentiometer (LED RPOT) and the photoresistor (PHOR). Additionally, there are 5 digital outputs with the LEDs, the LCD display that uses I2C to communicate, and an analog output from the DAC – Amplifier – Speaker path. The amplifier also has a potentiometer the controls the volume of the speaker output. Finally, the LDO circuit is the power management part of this project and connects to the VSYS pin on the Pico 2W. This allows the circuit to operate on battery power which makes it modular.

Altium Schematic

This Altium schematic was made after I designed the entire circuit and tested its functionality on the breadboard. Therefore, I had to switch to certain components to ensure that they could be placed on the PCB. For instance, I used a DIP LED IC instead of 5 individual LEDs to save space on the board, but the IC did not have any reliable footprint, so I changed it out for 5 individual LEDs. Moreover, I also had a separate button for each of the switches at first but opted to use the DIP Switch pack instead of different buttons. Moreover, I could find a switch pack that had a suitable footprint for the PCB, so the schematic below is what I finally settled on. However, there was a small issue with my design. I designed the circuit with the photoresistor connected to ground instead of the fixed resistor on the breadboard, but this is switched on the PCB. As a result, my design switched which modes corresponded to which darkness level. This was not a significant issue but was still something that could have been avoided through a more rigorous look at the schematic.

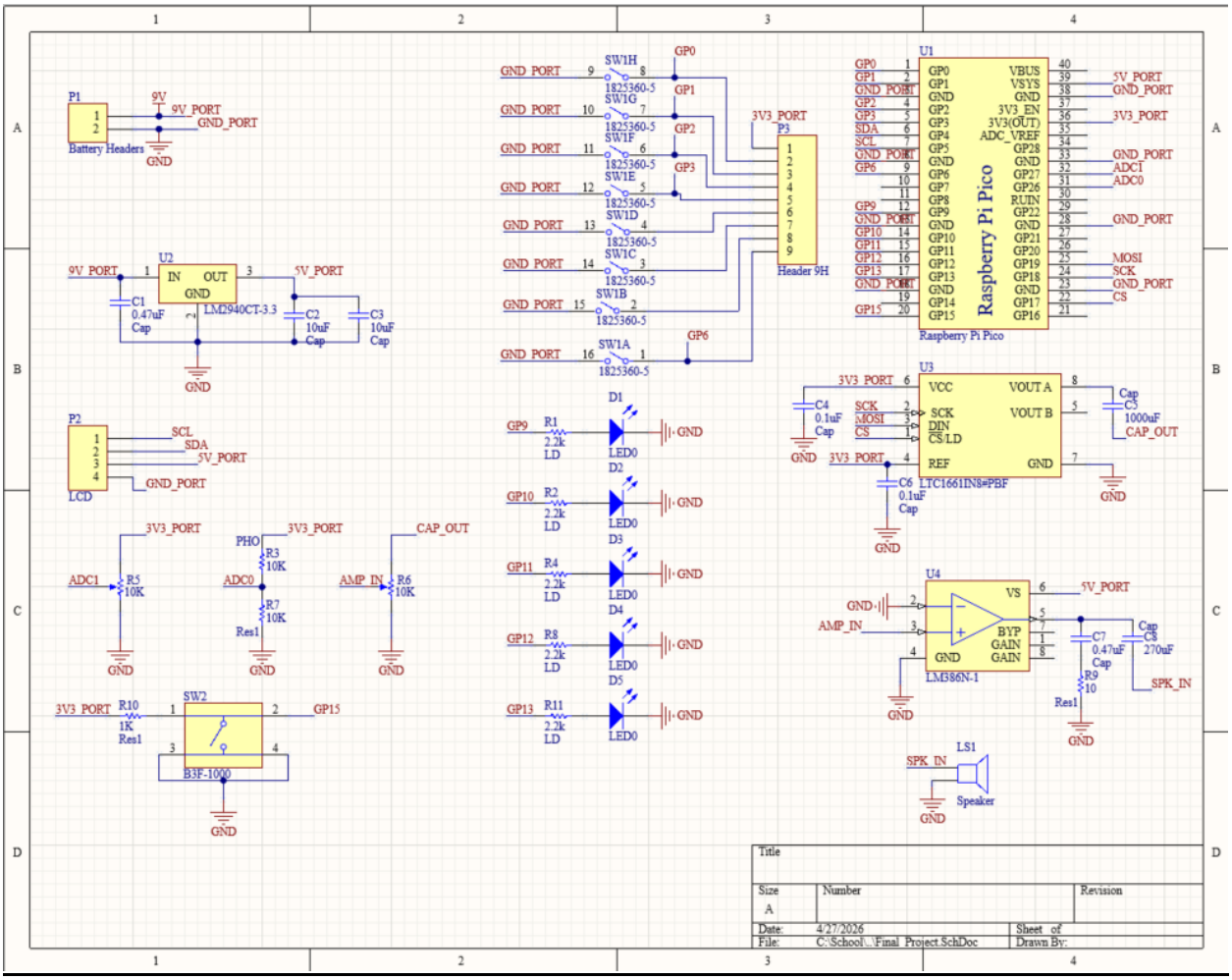

Figure 7: Altium Schematic

Altium Board Layout

There were 4 board layouts that I included in this section: the board planning layout, top layer layout, bottom layer layout, and full board layout. The board planning layout is just a rough diagram of every connection.

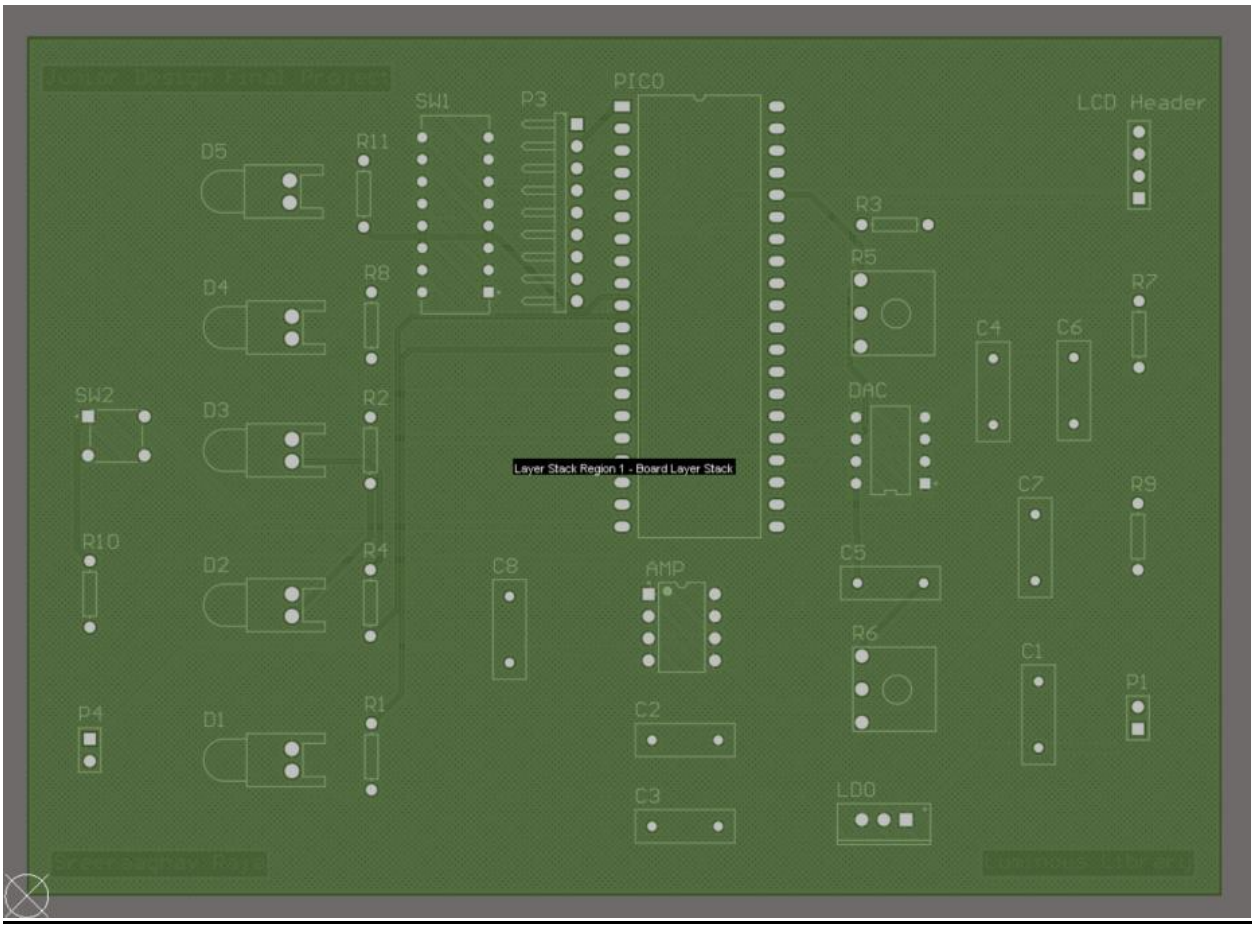

Figure 8: Board Planning Layout

The top layer layout has a polygon pour of 5V as the default. This is because the 5V line is connected to the most power-intensive components and stepping down from them is easier than stepping up to 5V. Usually the top layer polygon pour is determined on which value will minimize the most connections, but the layer could also be ground. Reducing the connections required in the PCB also reduces the fanout of the power lines, reducing the noise in the in design. The bottom layer is a ground pour which is effective because every component requires ground, so minimizing the fan-out of that connection is an optimal design choice for the PCB.

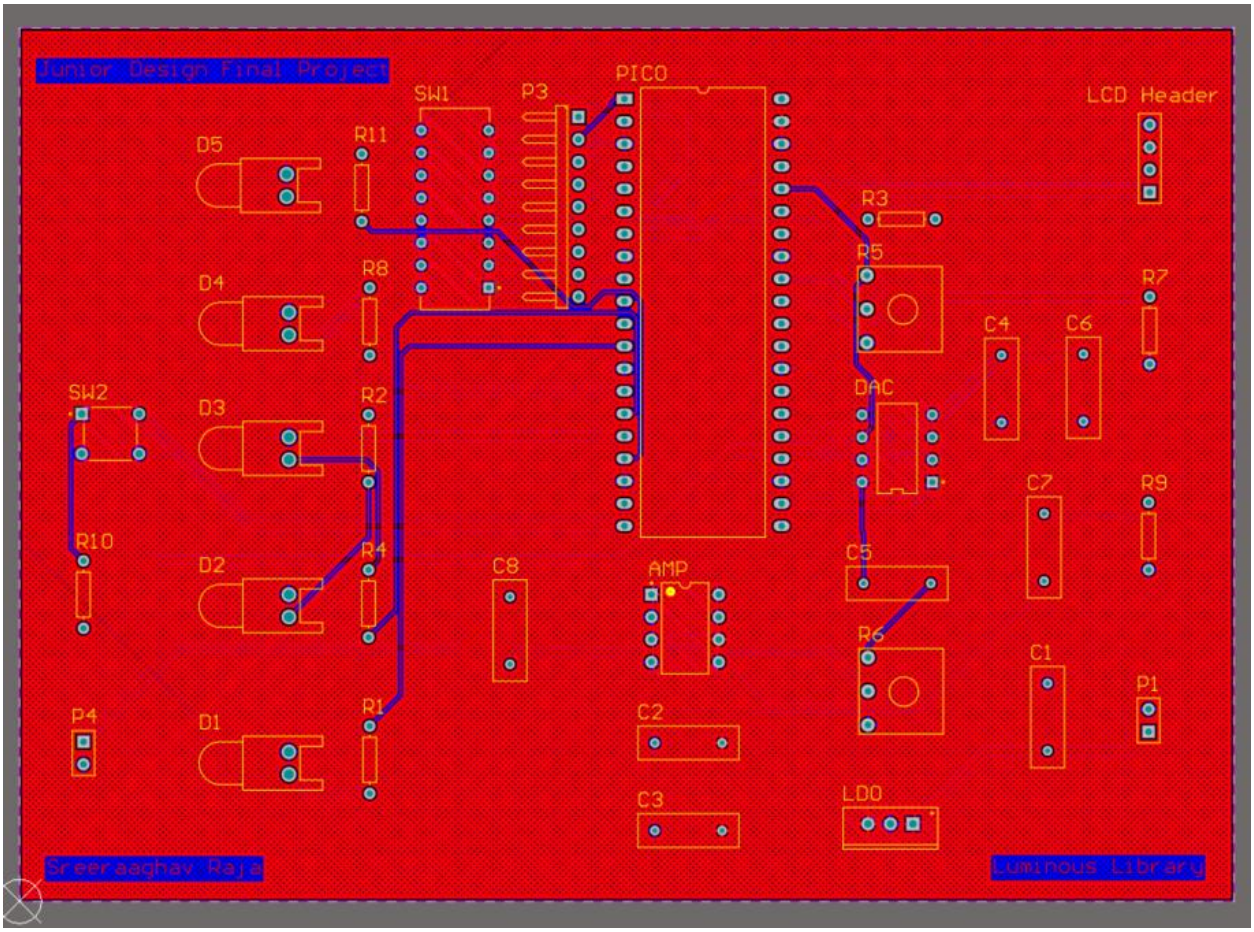

Figure 9: Top Layer PCB

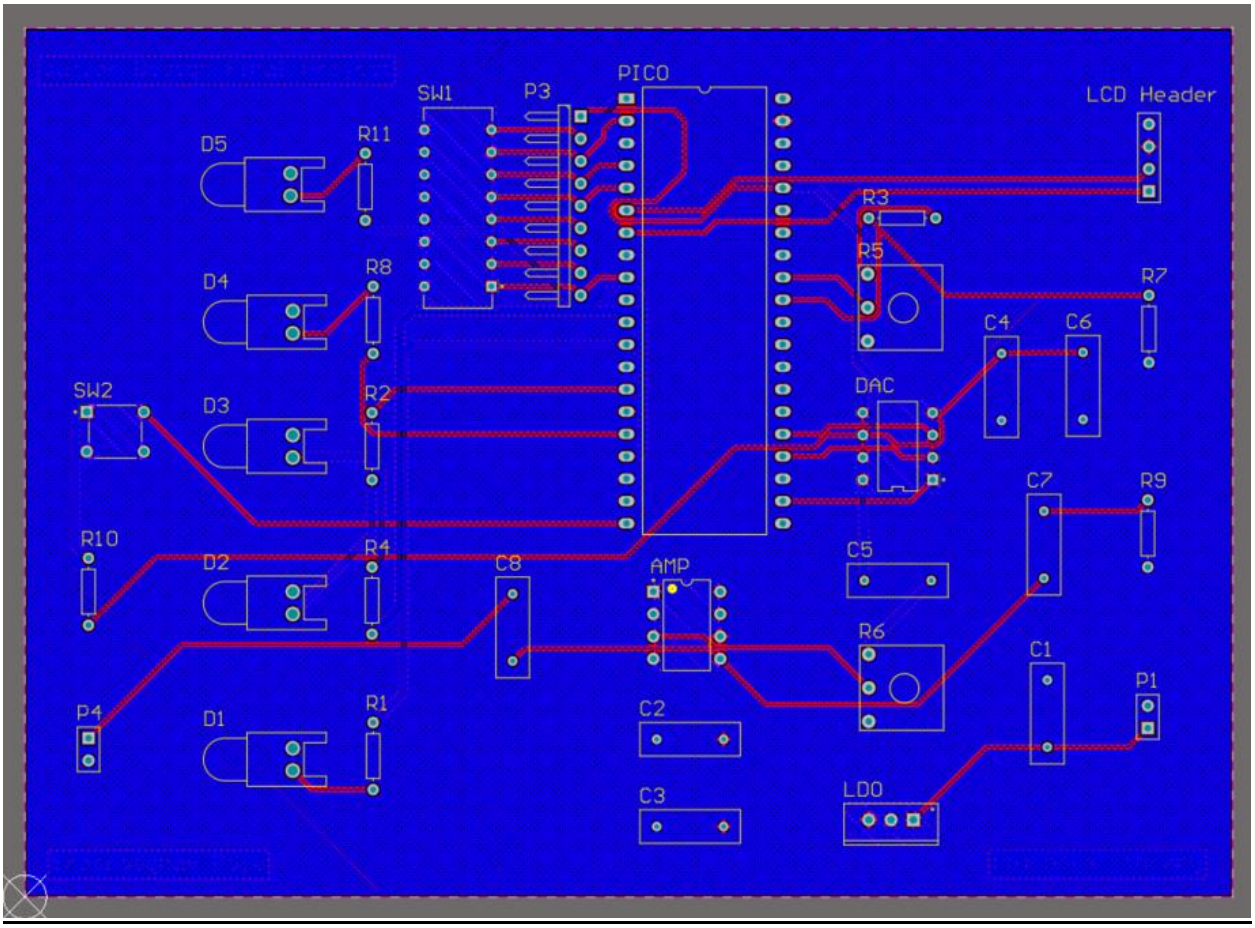

Figure 10: Bottom Layer PCB

The final board layout is the just how PCB would look with all the footprints and without it being assembled. The DAC, SW1, and LDO all have their corresponding components attached to them, but that was just part of their footprint. Moreover, the is relatively clean in terms of the components' placement, but the routing could have been improved as there are a lot of lines, especially near the LEDs, that are overlapping.

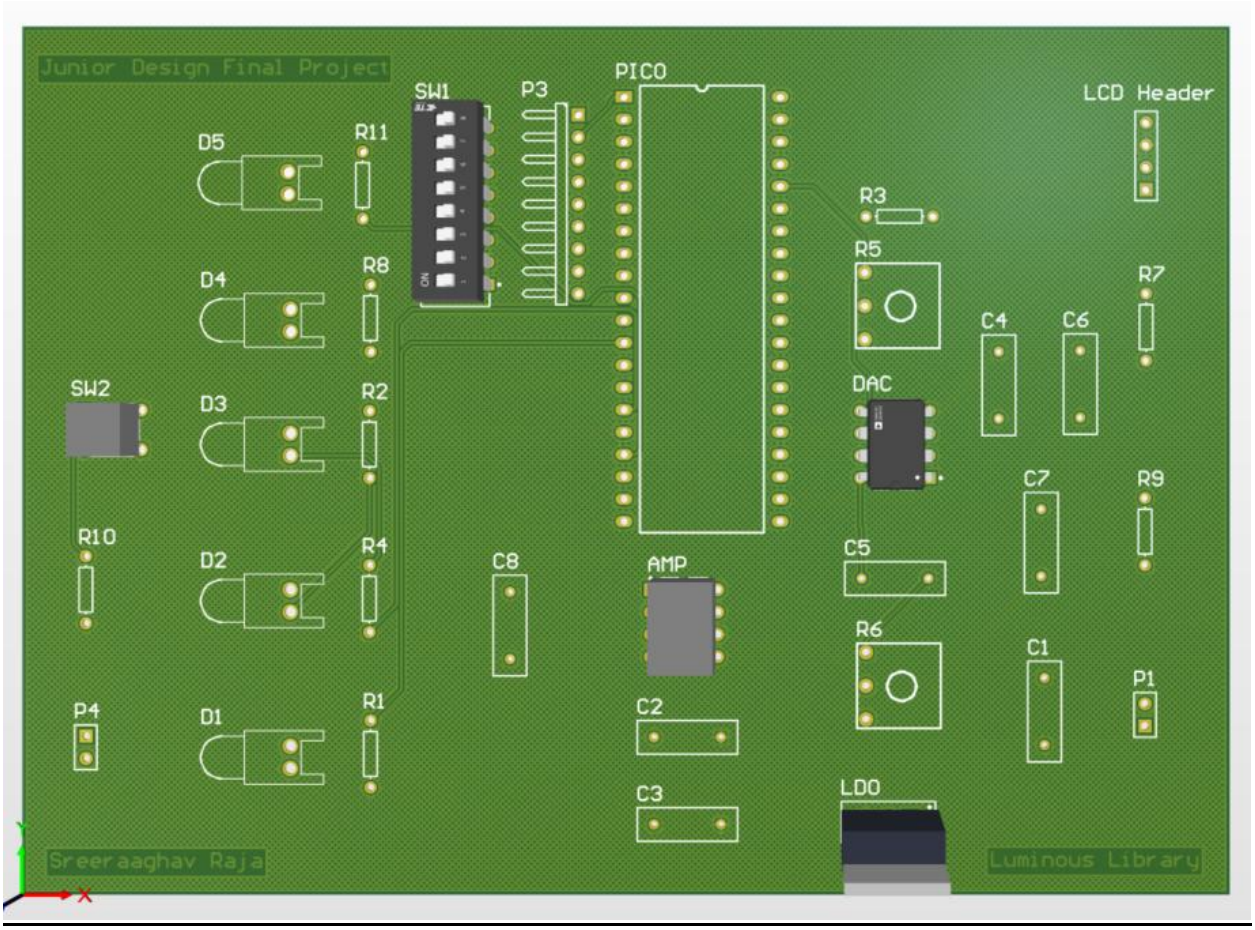

Figure 11: Full PCB Layout

Bill of Materials (BOM)

Component Name	Number of Items	Cost per Item ($)	Total Cost ($)
LTC1661	1	6.37*	6.37
LM386	1	1.05	1.05
Omron B3F-1000 Button	1	0.4	0.4
DIP Switch 8 Flush Slide	1	0.78	0.78
Resistor-SIP Package	1	0.2	0.2

2.2kΩ Resistor	5	0.21	1.05
10kΩ Resistor	1	0.22	0.22
100Ω Resistor	1	0.10	0.10
10Ω Resistor	1	0.14	0.14
Red LED	5	0.25	1.25
10µF Capacitor	2	0.70	1.40
0.47µF Capacitor	2	0.5	1.00
0.1µF Capacitor	2	0.75	1.50
10kΩ Potentiometer	2	0.70	1.40
1000µF Capacitor	1	0.19	0.19
470µF Capacitor	1	0.83	0.83
LDR/Photoresistor	1	0.95	0.95
16x2 LCD with I2C Backpack	1	3.92	3.92
LM2940CT LDO Voltage Regulator	1	1.94	1.94
Machine Pins	16	0.1	1.6
9V Battery Cap	1	0.5	0.5
Pico Socket Headers	2	0.625	1.25
Male to Female Jumper Cables	6	0.01	0.06

Header Pins	2	0.0475	0.095
Raspberry Pi Pico 2W	1	16.31	16.31
		Total Cost	44.505

The total price of the entire project minus the PCB is $44.505. The price of the PCB is exactly $17.00, so the total price of this project is $61.51. Moreover, I chose the price of individual components instead of the bulk price to account for the worst-case scenario of the price of this project. In other words, the price of this project is relatively cheap for the functionality. If there was a Wi-Fi component to this project, it could have been even more price effective.

Additionally, I chose vendors that had the lowest base price for most of these products as a means of offsetting not going for the bulk price for certain components. The priciest component in the entire project apart from the PCB was the Raspberry Pi Pico 2W which was fair because it is a relatively powerful microcontroller. Therefore, I could have reduced the price by using the Raspberry Pi Pico 2 which is $10 cheaper than the wireless version. Different vendors also had better prices for certain components or were the only sellers of that component. The Pico Socket Headers were sold by Adafruit, and they matched the exact specifications of the ones used in the project, so I added those to the BOM. However, Jameco was effective for smaller components like the header pins and capacitors. Mouser was effective when searching for resistors of different varieties, and Amazon was better for larger components like the Pico 2W and the LCD.

Results

PCB Pictures

The first image is an image of an empty PCB. This was just to show that the printed PCB matched the layout on Altium. Moreover, the biggest concern that I had was that the SIP resistor that I would use would match the sizing of the header P3. Thankfully, it did.

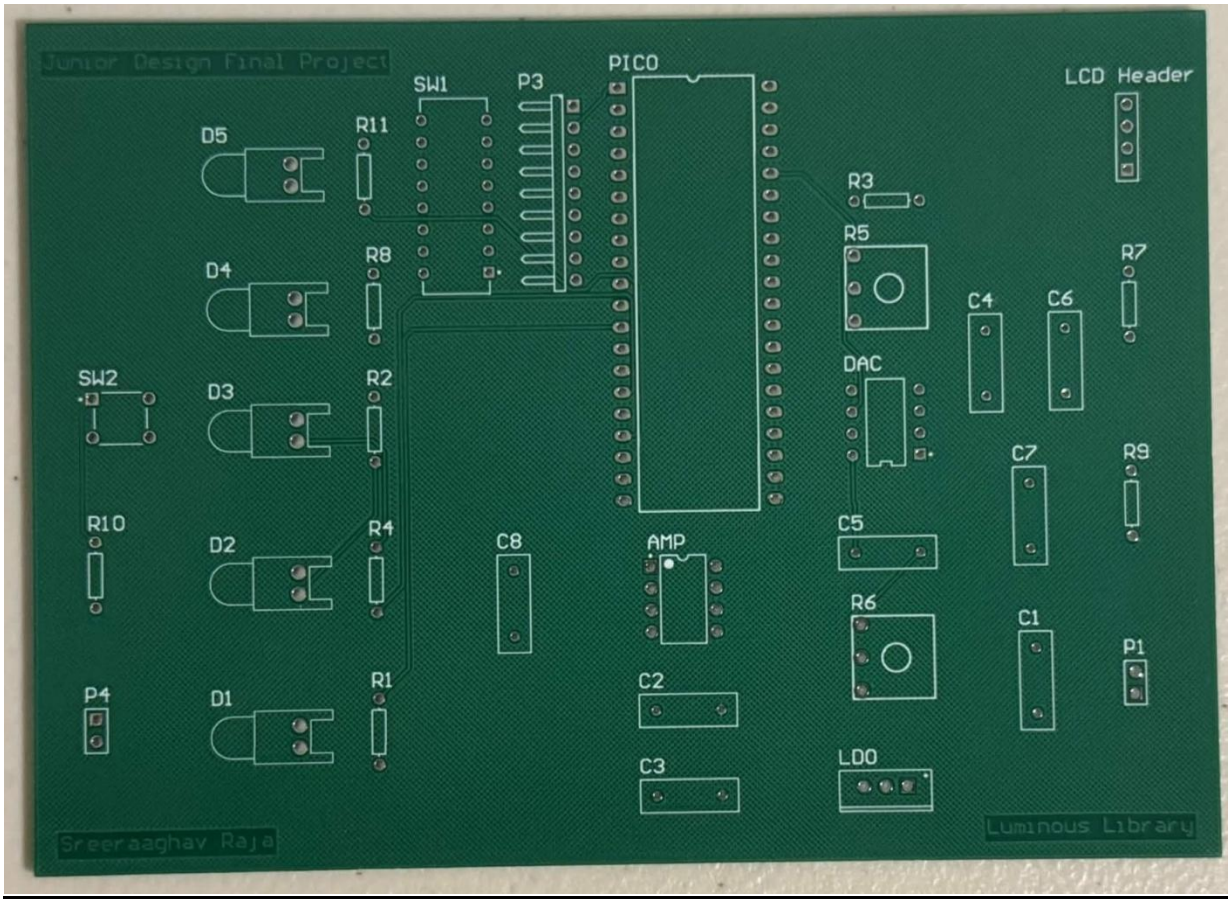

Figure 12: Empty PCB

Figure 11 shows the default state of the program alongside the completed PCB. The PCB is flipped but it does show every component soldered onto the board with a functional default state of the program.

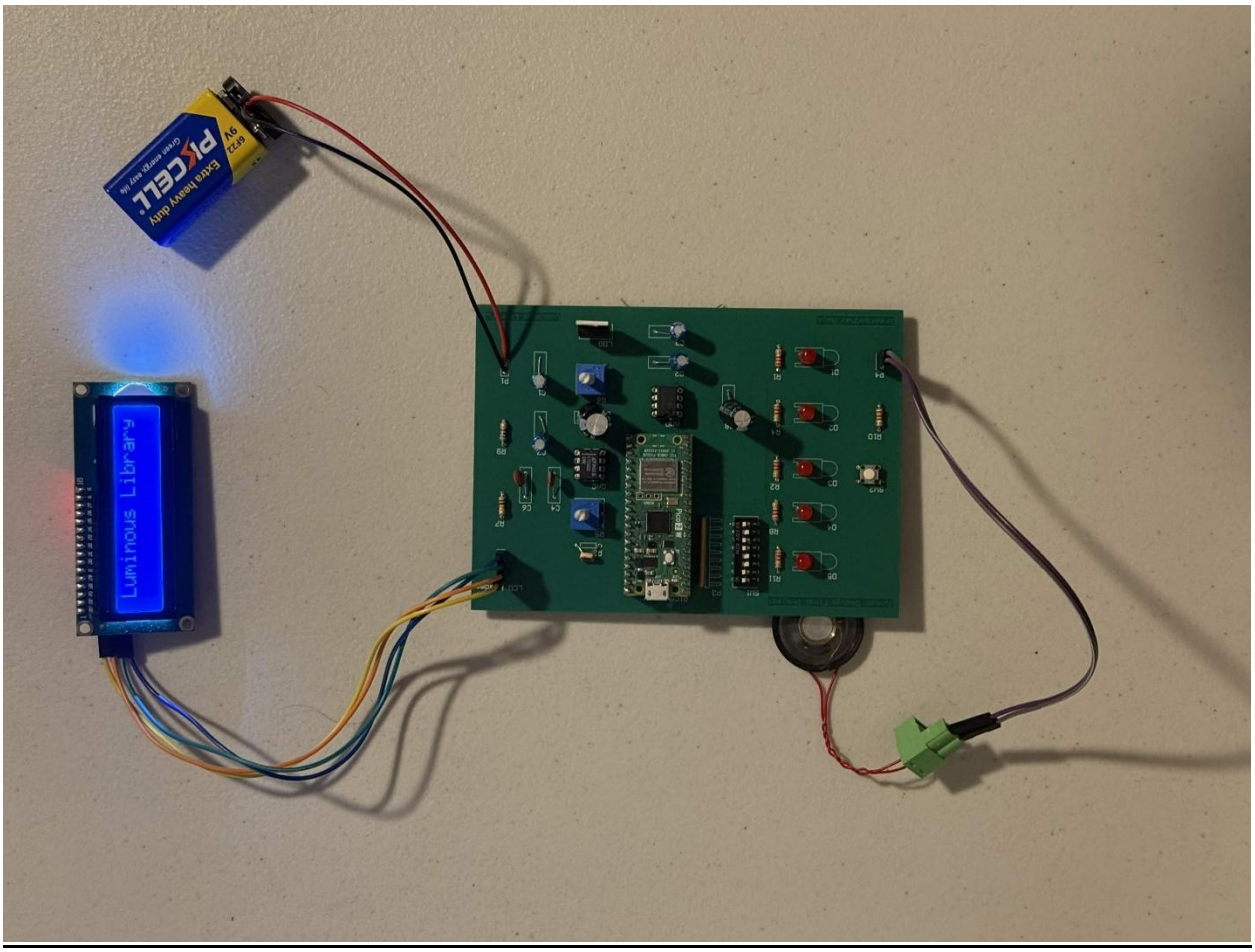

Figure 13: Full PCB

Documentation of System Function

State 1: Book Logging of 16 Default Genres

This state is the default log for the first 16 genres which are shown under Tasks.c in the appendix section of this report. Adventure is one of the values listed in the genres array.

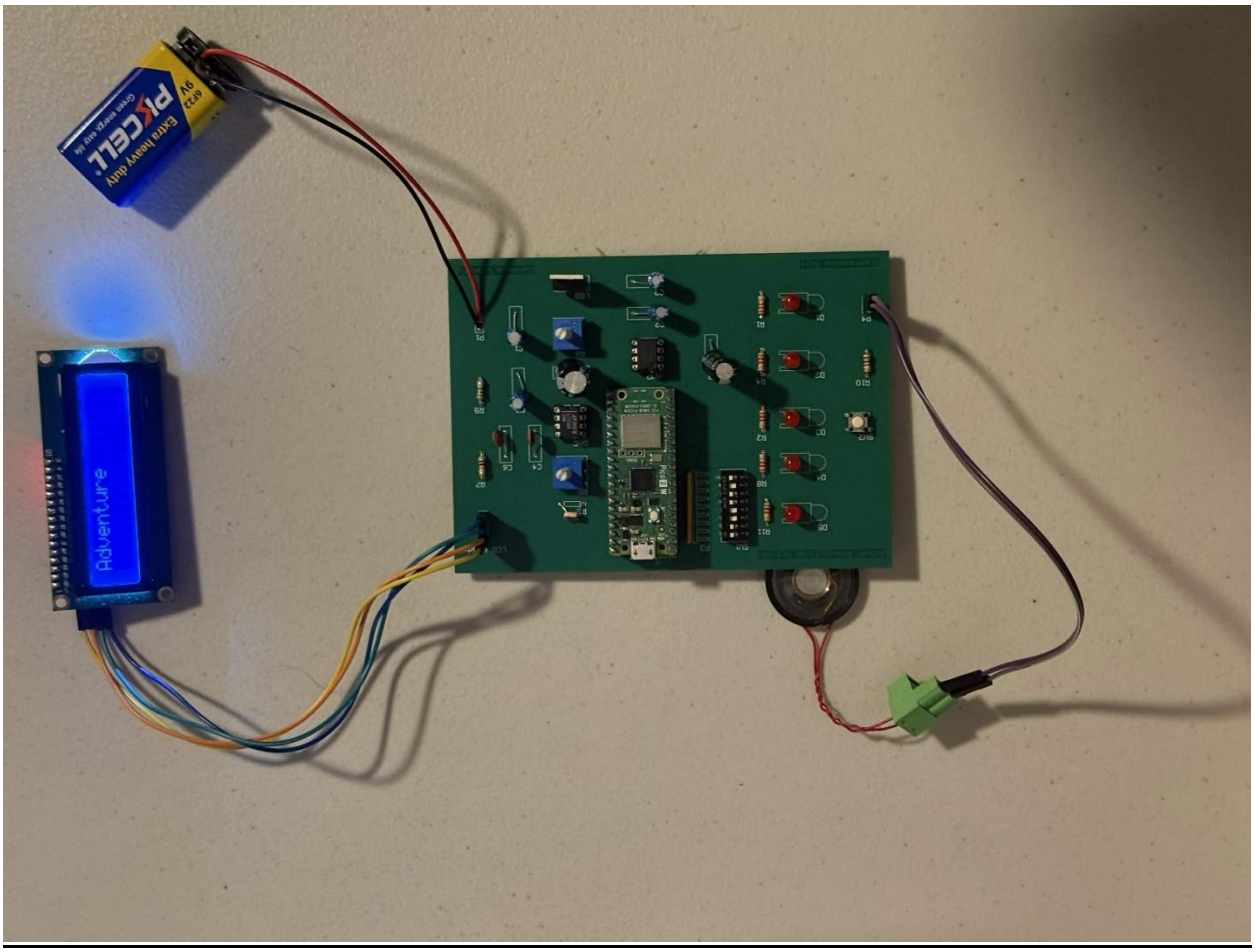

Figure 14: Default Log 1

This is the second part of the logging process which is progress logging. It is important to note that two of the LEDs do not function properly, so there may have been either a soldering issue or flipped leads on the LEDs when inserted into the PCB.

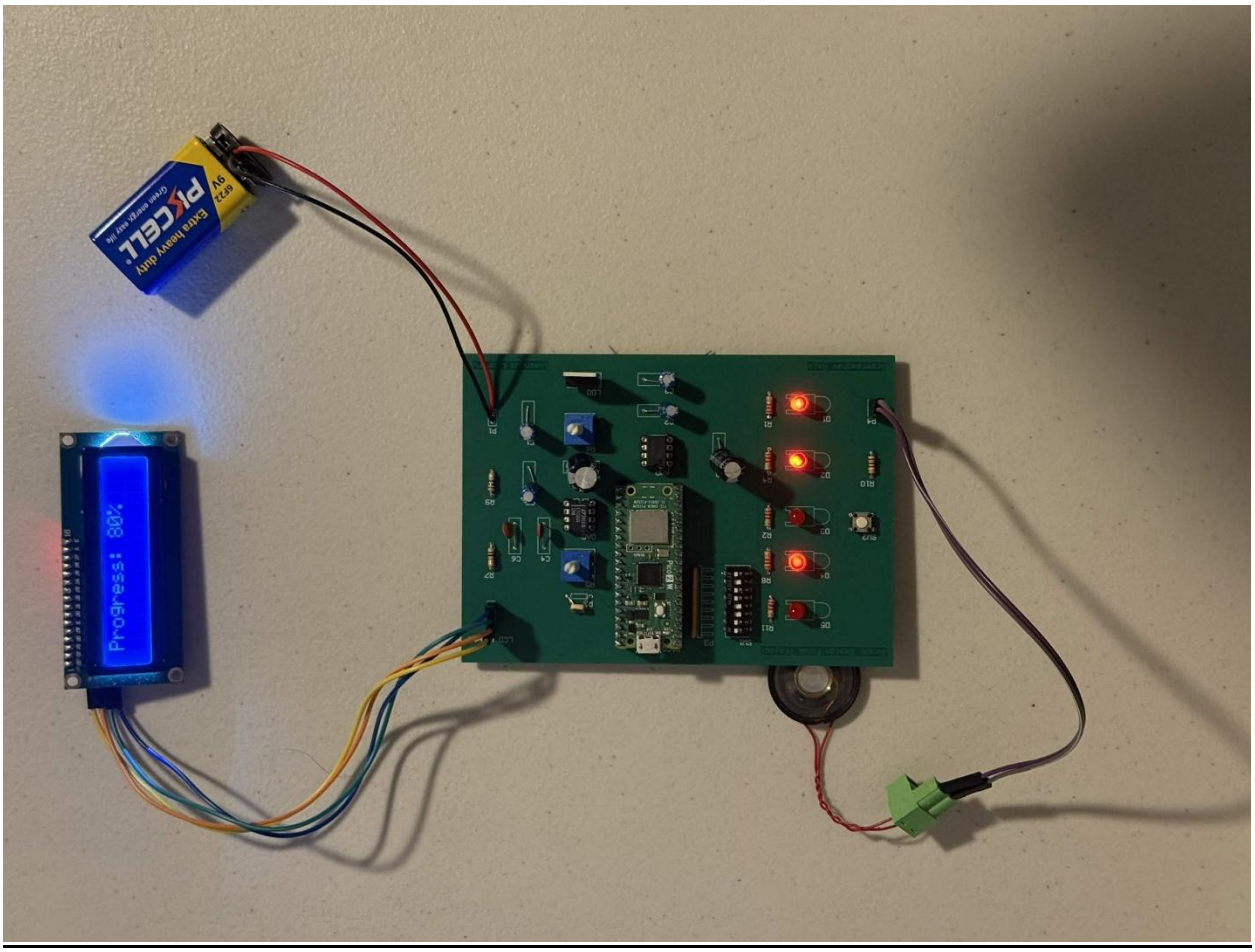

Figure 15: Default Log 2

This shows that progress can change when turning the pot at R5.

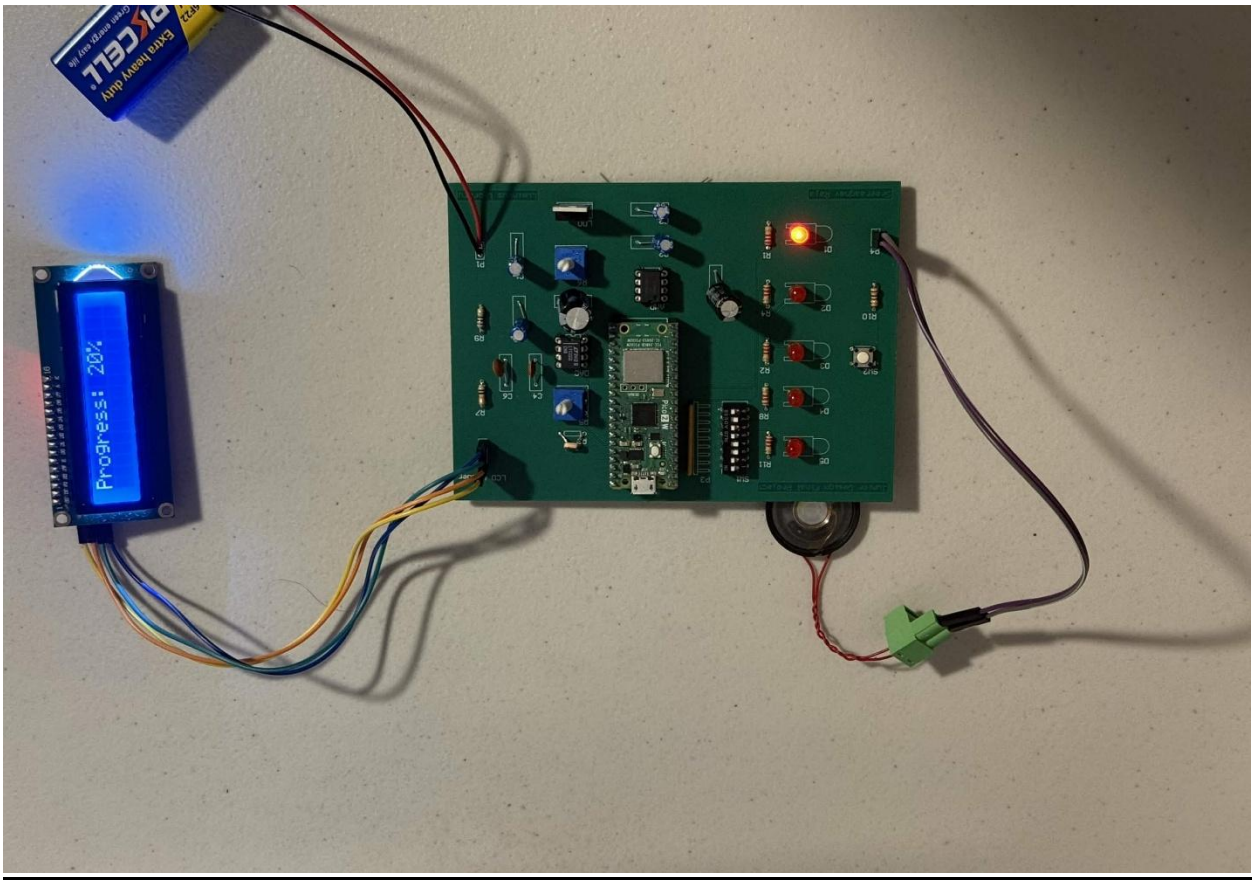

Figure 16: Default Log 3

This is a completed logging process. If there was video, the chime would be playing.

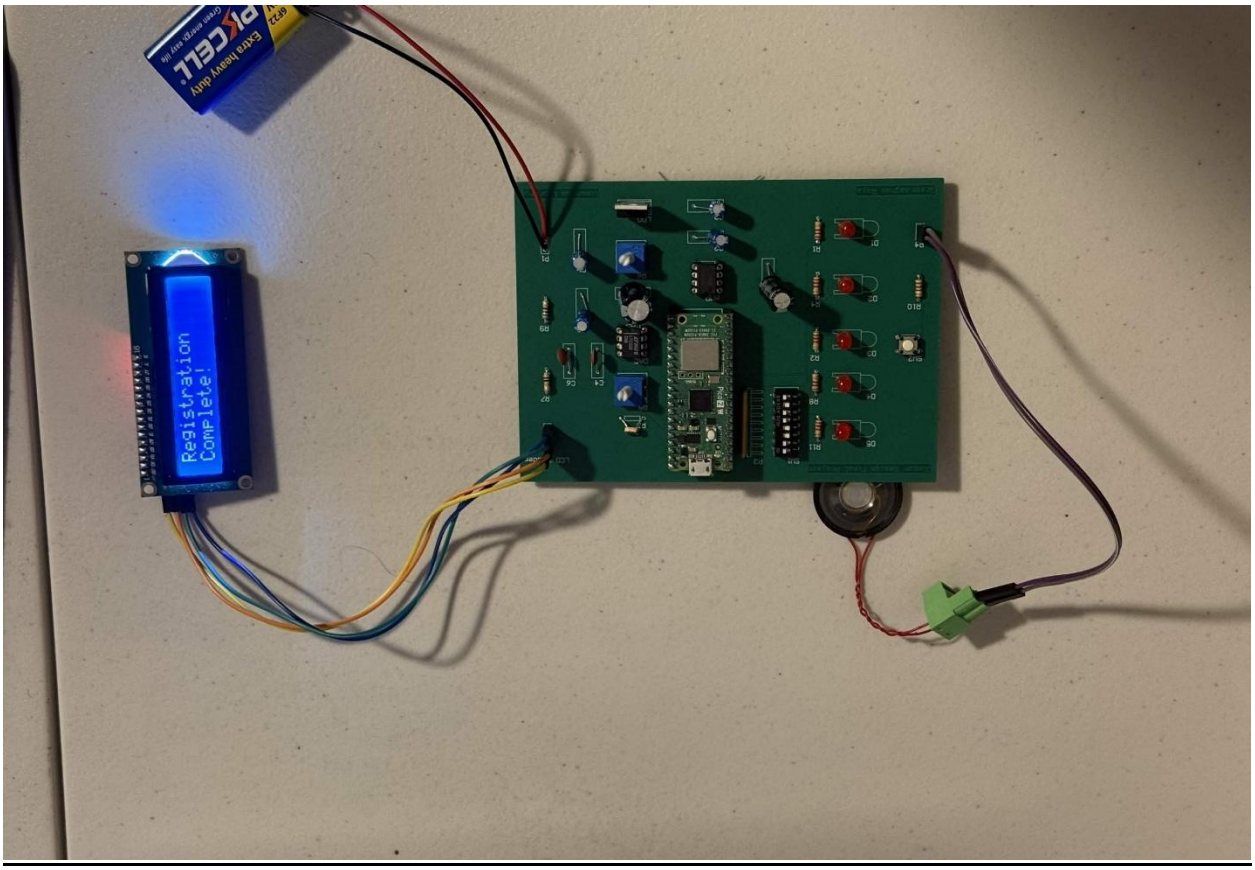

Figure 17: Default Log 4

State 2: Book Logging of 16 New Genres

This state is reached by completely covering the photoresistor which would let the ADC read a value of 1.67V. These genres are referred to as **bright_genres** in the Tasks.c file in the appendix, but the mishap with the PCB caused them to be the **dark_genres** instead. Music and Nature are both in the **bright_genres** array in Tasks.c so this shows the proper functionality of this state.

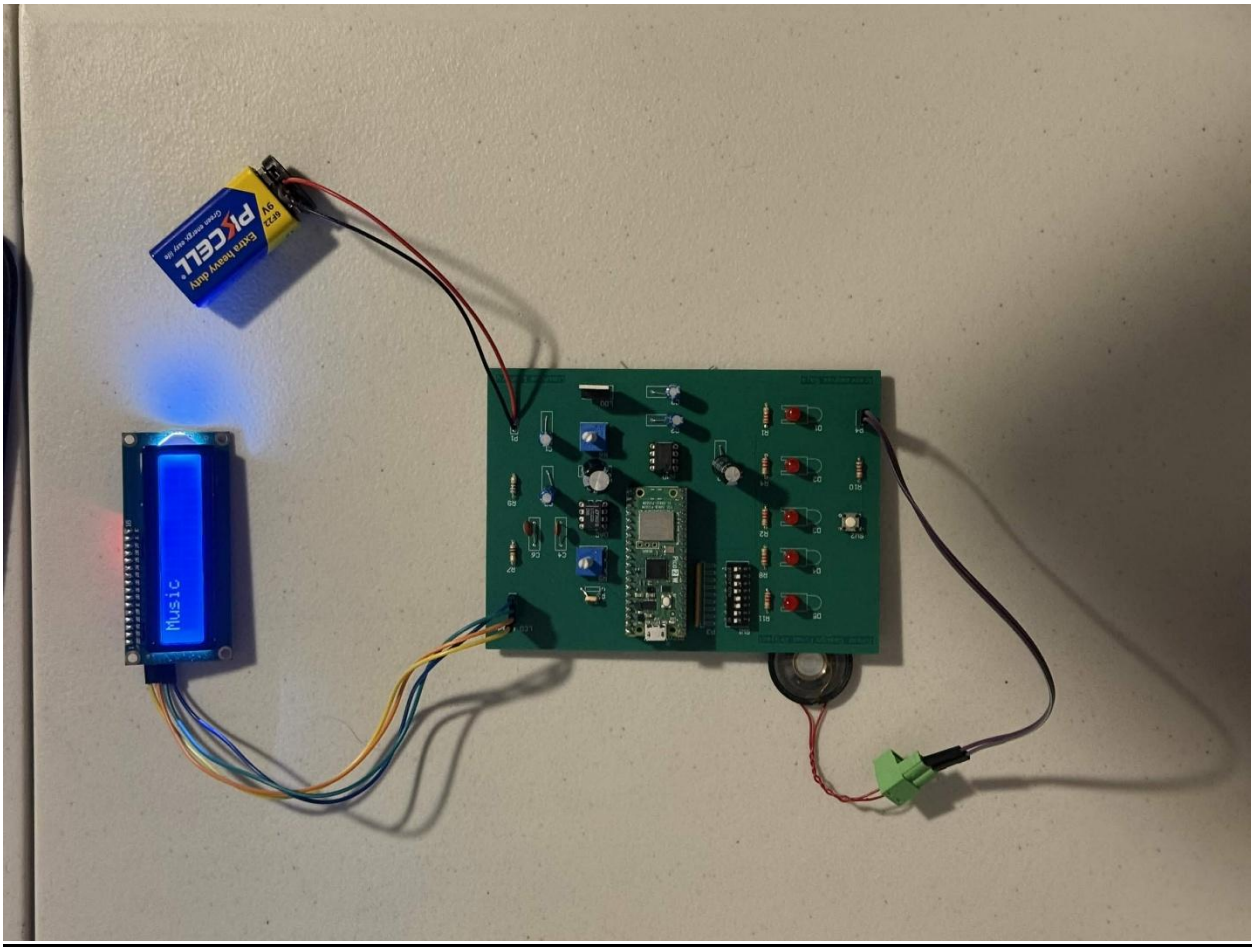

Figure 18: New Log 1

This is another genre in the **bright_genres** array which is achieved by changing the values of the switch pins.

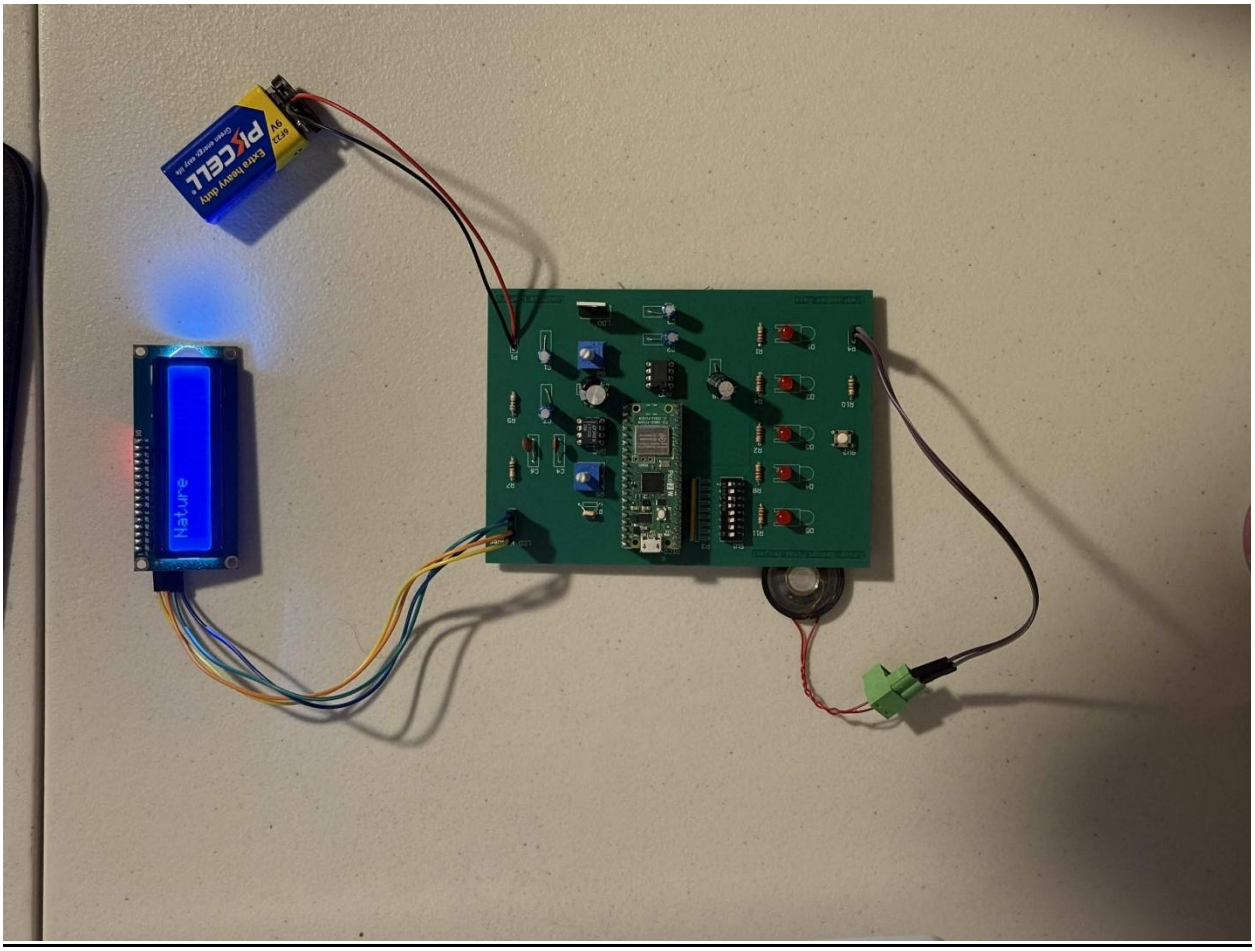

Figure 19: New Log 2

This shows proper logging of the progress which is changed by turning the potentiometer, but this was also shown earlier with state 1.

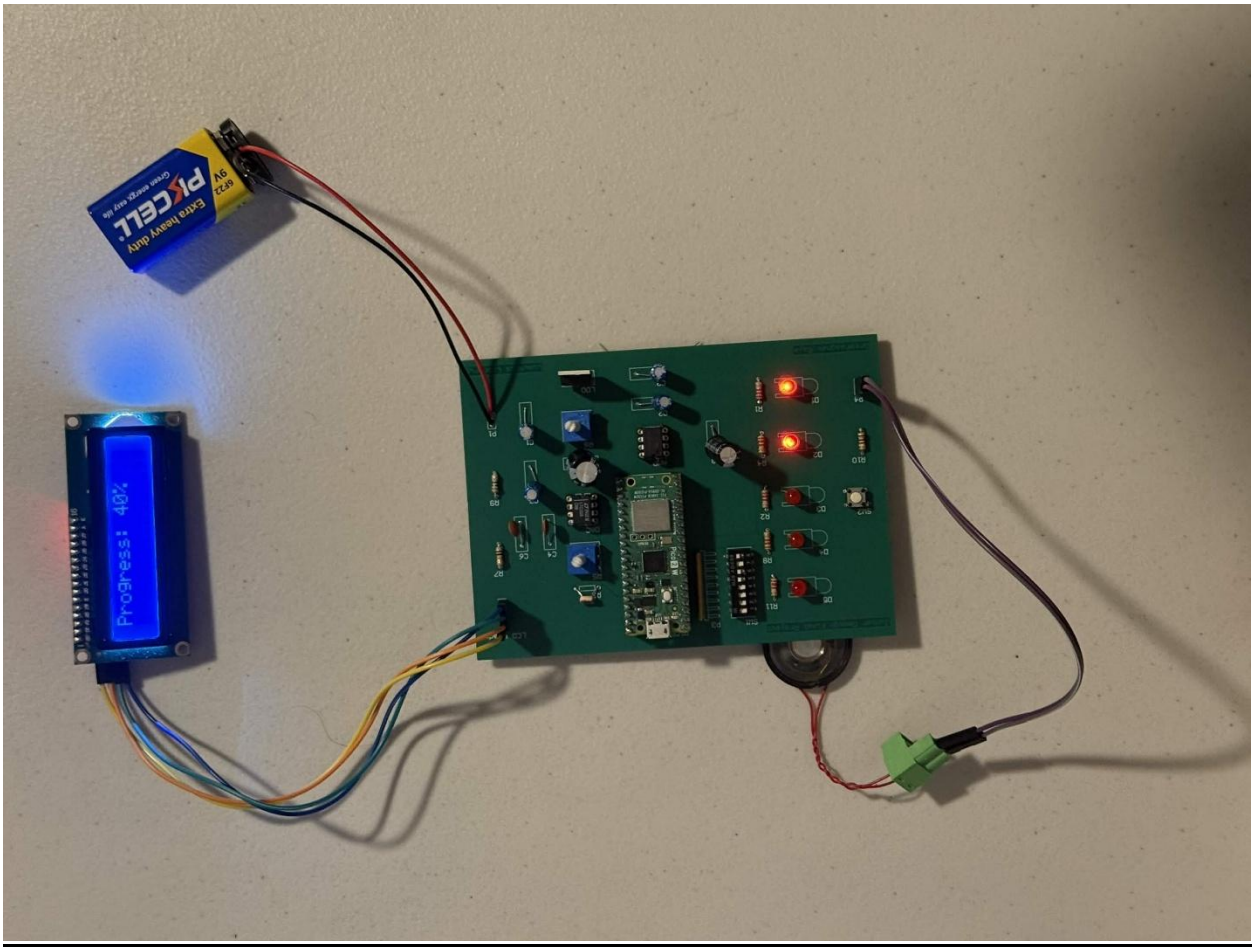

Figure 20: New Log 3

This shows the proper completion / registration of the book logging process which will also play the chime.

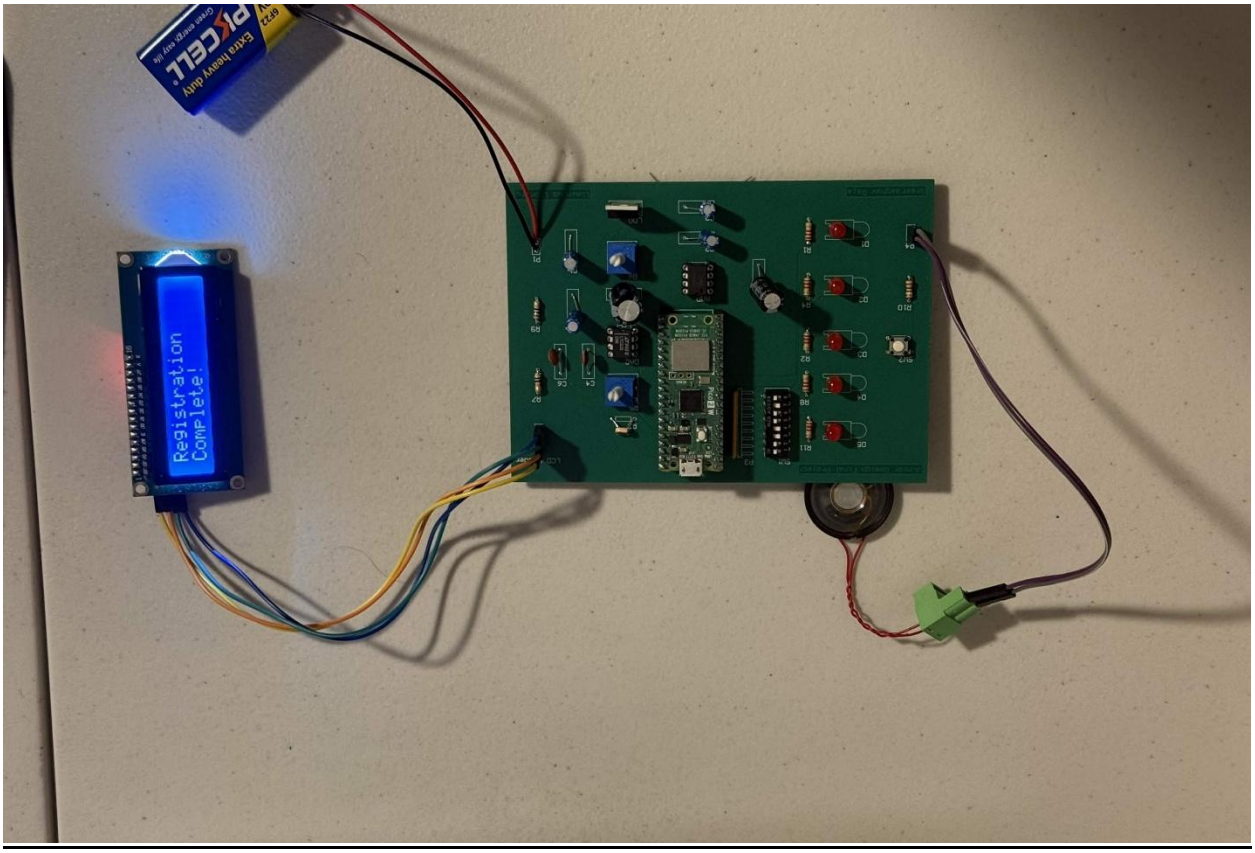

Figure 21: New Log 4

State 3: Recommendation State

The recommendation state is achieved by exposing the photoresistor to light where the ADC would read a value of 3.3V. Moreover, this program gives a recommendation for each of the 16 default genres. So, for Classics it would recommend *1984* and for Historical it would recommend *All Quiet on the Western Front*. For this case, the program recommends *Heated Rivalry* for LGBTQ+. There is no registration for this state, so this is the only aspect of this state.

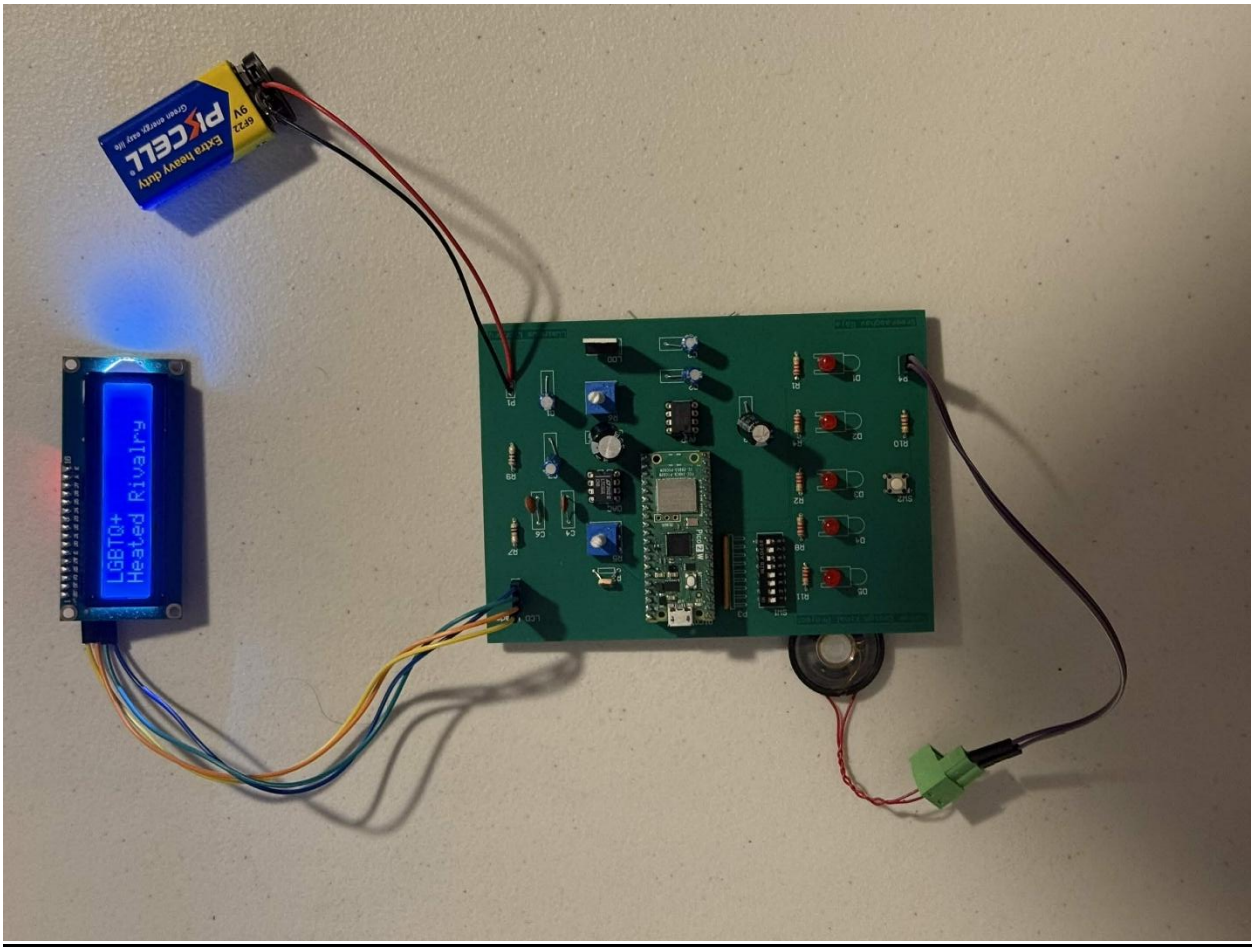

Figure 22: Recommendation Log 1

State 4: Logs

The final state is an extra state that does not correspond to the photoresistor. This state plays back all the previously logged book genres and their progress. However, if there is nothing logged yet, the program displays “No Logs Found.”

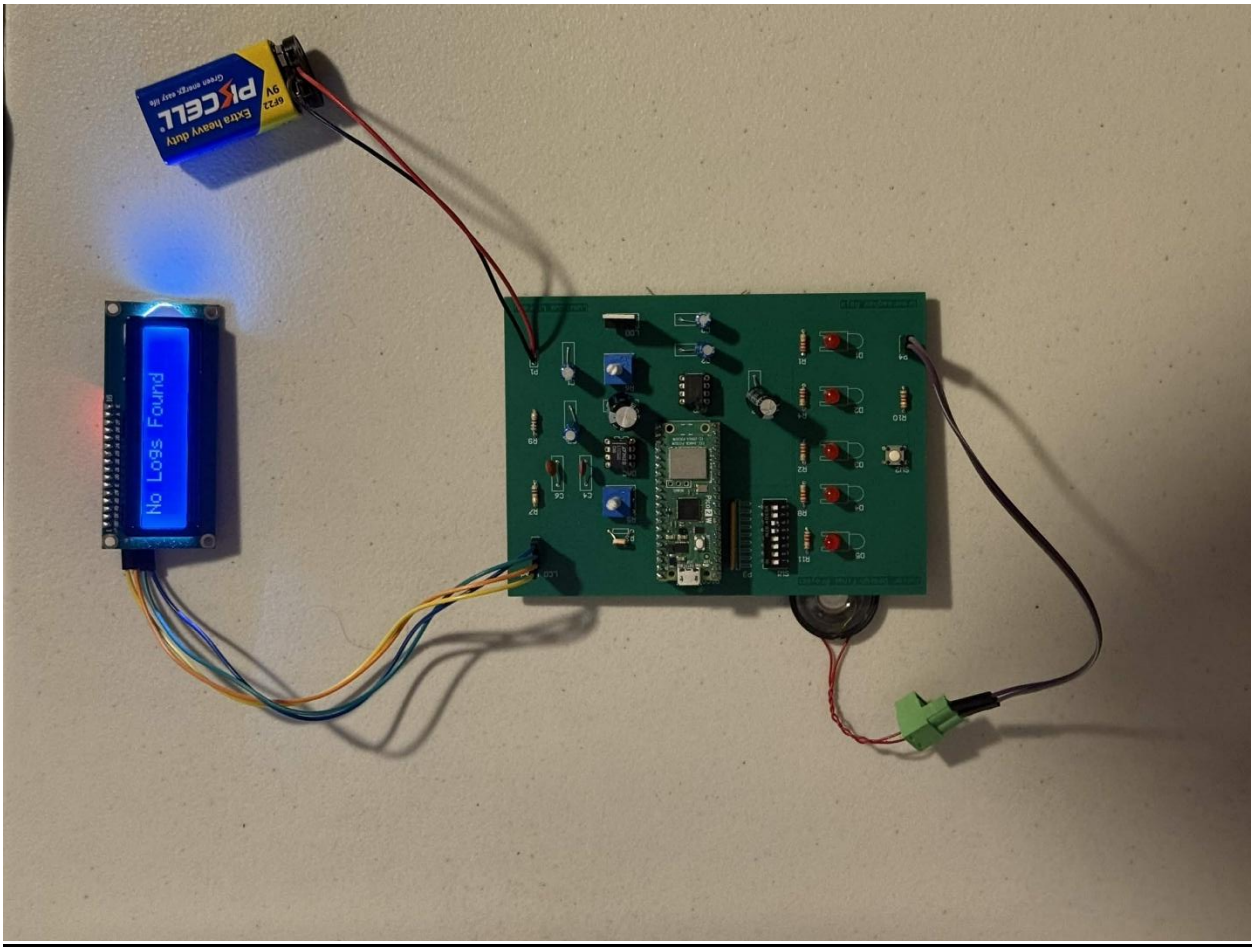

Figure 23: Logs 1

This shows one of the previous genres and logs. This is an instance of the logging Adventure with 20% progress.

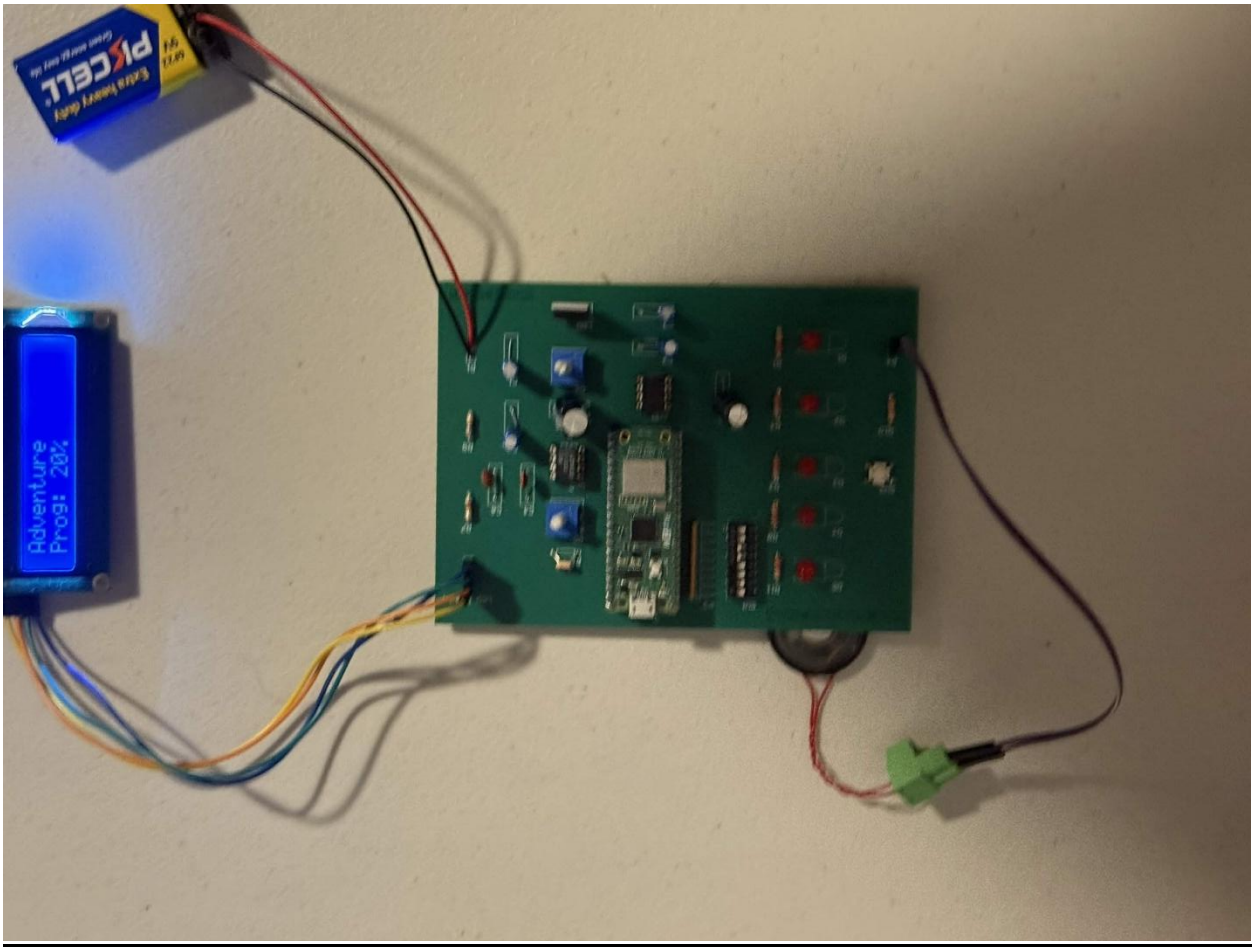

Figure 24: Logs 2

This is an example of logging Nature with 40% progress.

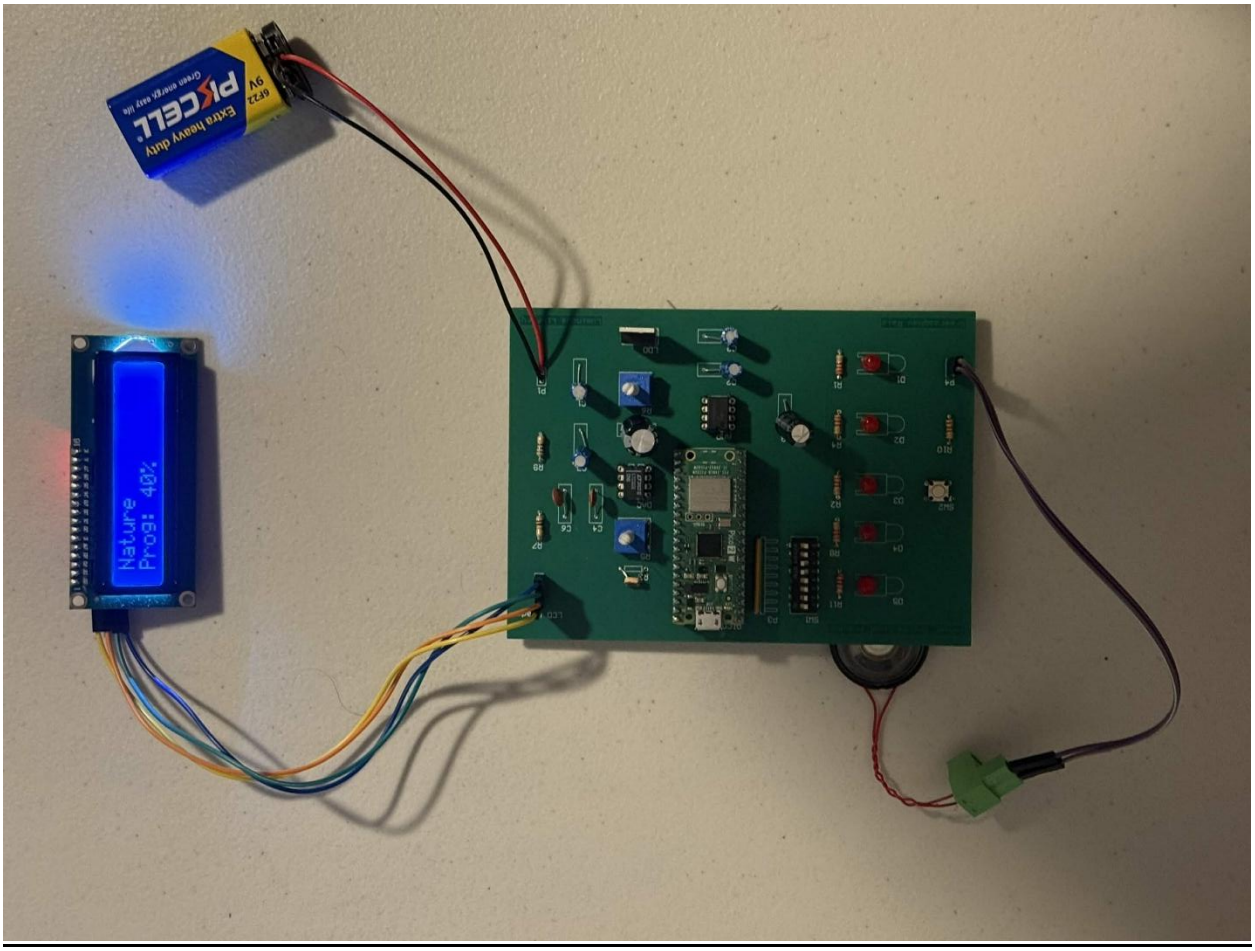

Figure 25: Logs 3

Oscilloscope Output

The oscilloscope output is comprised of two readings: one directly from the DAC output and one from the Speaker. The DAC output is shown below and is a very clean sine wave with a set period. The voltage is 3.3V and that is connected to the 270µF capacitor which connects to a potentiometer that tunes the amplitude for the amplifier.

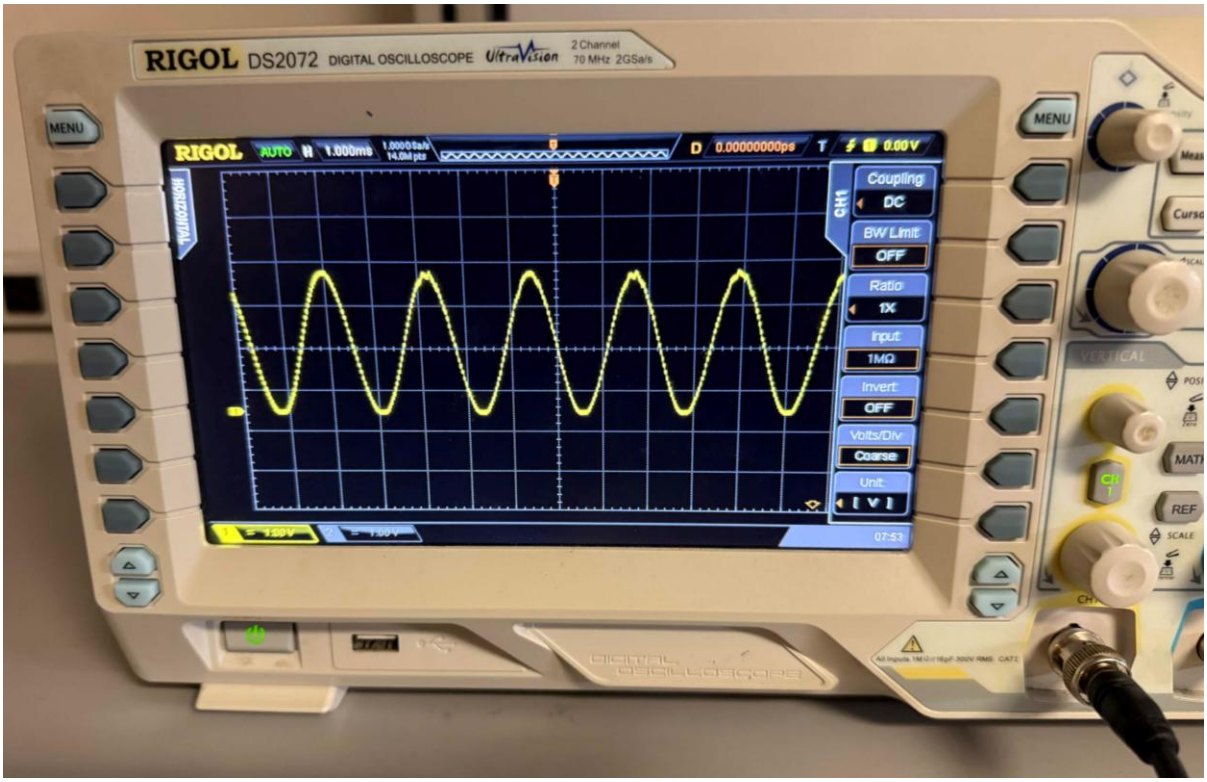

Figure 26: DAC Output

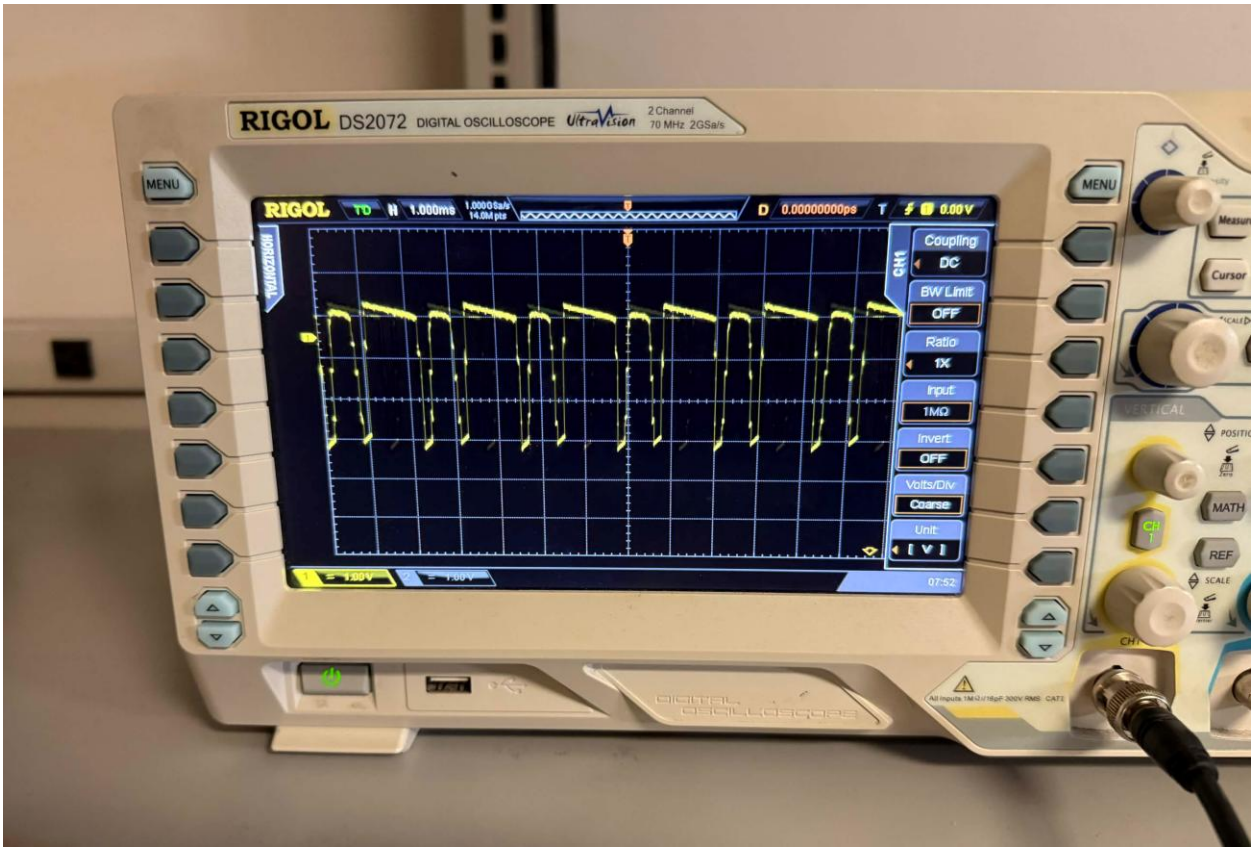

Figure 27: Speaker Output

The output of the speaker is still a sinusoidal signal, but it is much less clean. The signal becomes more of a square wave because of the amplifier. This was the case for the amplifier lab as well as the sinusoidal signal becomes a square wave, so this is the correct functionality for this lab. The differences in the output could be caused by the 10Ω resistor that drives current away from the speaker or with the internal configuration of the LM386. There were many faulty LM386 ICs, so this may have been one that was faulty. Nevertheless, the chime output was clean, and the feature worked as expected.

Discussion/Conclusion

The goal of my system was to integrate my love of reading into a design that had a fun and interesting user experience. The primary use case for this system is to have a gamified way of logging the progress when reading a book when the user does not want to expose themselves to the harsh light of electronic devices. For instance, if they were reading late at night, they could log their progress with my devices.

There were many challenges that I faced when I built this project, but the greatest challenge was setting up FreeRTOS for the Raspberry Pi Pico 2W. There is a lot of background setup required when programming the Pico 2 in C because of MakeFiles and the Pico Software Development Kit (SDK), but FreeRTOS has been by far the hardest thing to set up for this project. There were also challenges with interfacing with the DAC since I did that lab in MicroPython. Therefore, I had to use what I did in MicroPython as a template for the DAC in C, but that took half the time it took to set up the FreeRTOS environment.

Overall, I learned that the utilizing audio will always be a daunting task no matter the project, so I will try to improve my own understanding of audio engineering by creating more audio-based projects. Moreover, the hardware functionality was perfect in testing but not with the PCB, so I would likely get a DIP LED pack instead of individual LED for each value. The software functionality was not ideal because I couldn't integrate Wi-Fi into this project. That is something that I would like to expand on in the future with this exact project since I already have the code. This project was exciting to make, and I hope to create and test other projects in the future too.

References

[1] Seeed Studio, "Potentiometer: Functions, Types and Applications," Oct. 22, 2019. [Online]. Available: https://www.seeedstudio.com/blog/2019/10/22/potentiometer-functions-types-and-applications/. [Accessed: Apr. 27, 2026].

[2] EE Power, "Photoresistor (LDR)," *Resistor Guide*. [Online]. Available: https://eepower.com/resistor-guide/resistor-types/photo-resistor/. [Accessed: Apr. 27, 2026].

[3] Physics and Radio-Electronics, "What is a photoresistor (LDR)? Construction, Working and Applications." [Online]. Available: https://www.physics-and-radio-electronics.com/electronic-devices-and-circuits/passive-components/resistors/photoresistor.html. [Accessed: Apr. 27, 2026].

[4] Circuit Crush, "The Ultimate Guide to Digital-to-Analog Converters (DACs)." [Online]. Available: https://circuitcrush.com/digital-analog-converters-tutorial/. [Accessed: Apr. 27, 2026].

[5] Analog Devices, "Introduction to SPI Interface," *Analog Dialogue*. [Online]. Available: https://www.analog.com/en/resources/analog-dialogue/articles/introduction-to-spi-interface.html. [Accessed: Apr. 27, 2026].

[6] SparkFun Electronics, "Pull-up Resistors." [Online]. Available: https://learn.sparkfun.com/tutorials/pull-up-resistors/all. [Accessed: Apr. 27, 2026].

[7] CircuitBread, "What are the forward voltages of different LEDs?," *EE FAQ*. [Online]. Available: https://www.circuitbread.com/ee-faq/the-forward-voltages-of-different-leds. [Accessed: Apr. 27, 2026].

[8] FreeRTOS, "RTOS Fundamentals," *FreeRTOS Documentation*. [Online]. Available: https://www.freertos.org/Documentation/01-FreeRTOS-quick-start/01-Beginners-guide/01-RTOS-fundamentals. [Accessed: Apr. 27, 2026].

[9] GeeksforGeeks, "Difference between Mutex and Semaphore," *Operating Systems*. [Online]. Available: https://www.geeksforgeeks.org/operating-systems/mutex-vs-semaphore/. [Accessed: Apr. 27, 2026].

Appendix

Header Files:

FreeRTOSConfig.h

```
#ifndef FREERTOS_CONFIG_H
#define FREERTOS_CONFIG_H

/* Hardware Specific Settings for RP2350 */
#define configCPU_CLOCK_HZ              ( 150000000UL )
#define configTICK_RATE_HZ              ( ( TickType_t ) 1000 )
#define configMAX_PRIORITIES            ( 5 )
#define configMINIMAL_STACK_SIZE        ( 256 )

/* RP2350 Specific Port Settings - REQUIRED FOR CORTEX-M33 */
#define configENABLE_FPU                1
#define configENABLE_MPU                0
#define configENABLE_TRUSTZONE          0
#define configRUN_FREERTOS_SECURE_ONLY  1
#define configMAX_API_CALL_INTERRUPT_PRIORITY   16
#define configMAX_SYSCALL_INTERRUPT_PRIORITY    16

/* SMP / Multi-core Settings */
#define configNUMBER_OF_CORES           2
#define configUSE_PASSIVE_IDLE_HOOK     0
#define configTICK_CORE                 0
#define configRUN_MULTIPLE_PRIORITIES   1
#define configUSE_CORE_AFFINITY         1
/* portCRITICAL_NESTING_IN_TCB removed - port defines this as 0 */
#define configUSE_TASK_PREEMPTION_DISABLE    0

/* SMP Spinlock configuration */
#define configSMP_SPINLOCK_0            16
#define configSMP_SPINLOCK_1            17

/* Memory Management */
#define configSUPPORT_DYNAMIC_ALLOCATION    1
#define configTOTAL_HEAP_SIZE           ( 64 * 1024 )
#define configTIMER_TASK_STACK_DEPTH    ( configMINIMAL_STACK_SIZE * 2 )

/* Task Scheduling */
#define configUSE_PREEMPTION            1
#define configUSE_TIME_SLICING          1
#define configUSE_IDLE_HOOK             0
#define configUSE_TICK_HOOK             0
```

```
#define configUSE_MUTEXES                1
#define configUSE_RECURSIVE_MUTEXES      1
#define configUSE_COUNTING_SEMAPHORES    1

/* Software Timers */
#define configUSE_TIMERS                 1
#define configTIMER_TASK_PRIORITY        ( 3 )
#define configTIMER_QUEUE_LENGTH         ( 10 )

/* API Function Inclusion */
#define INCLUDE_vTaskPrioritySet         1
#define INCLUDE_vTaskDelay               1
#define INCLUDE_vTaskDelete              1
#define INCLUDE_vTaskSuspend             1
#define INCLUDE_xQueueGetMutexHolder     1
#define INCLUDE_xTimerPendFunctionCall   1

/* Misc */
#define configALLOW_MUTEX_SELF_RECURSION  1
#define configUSE_16_BIT_TICKS           0
#define configIDLE_SHOULD_YIELD          1

#define configASSERT( x ) if( ( x ) == 0 ) { portDISABLE_INTERRUPTS(); for( ;; ); }

#endif /* FREERTOS_CONFIG_H */
```

Tasks.h

```
#ifndef _TASKS_H
#define _TASKS_H

#include <stdio.h>
#include "pico/stdlib.h"
#include "FreeRTOSConfig.h"
#include "FreeRTOS.h"

#define MAX_MAP_SIZE 256

// make a struct for the stored data
typedef struct{
   char genre[16];
   int progress;
} LibraryLog;
```

```
// initialize a list of all the data;
extern LibraryLog DataMap[MAX_MAP_SIZE];

void Progress_Task(void *pvParameters);
void button_isr(uint gpio, uint32_t events);
void Button_Task(void *pvParameters);
void Switch_Task(void *pvParameters);
void Chime_Task(void *pvParameters);
void Photo_Task(void *pvParameters);
void Log_Task(void *pvParameters);

extern TaskHandle_t xProgressHandle;
extern TaskHandle_t xButtonHandle;
extern TaskHandle_t xSwitchHandle;

#endif /*_TASKS_H*/
```

Semaphores.h

```
#ifndef _SEMAPHORES_H
#define _SEMAPHORES_H

#include "FreeRTOS.h"
#include "pico/stdlib.h"
#include "semphr.h"

extern SemaphoreHandle_t xButtonSemaphore;
extern SemaphoreHandle_t xLedMutex;
extern SemaphoreHandle_t xChimeSemaphore;
extern SemaphoreHandle_t xADCMutex;


#endif /*_SEMAPHORES_H */
```

LCD_Init.h

```
#ifndef _LCD_INIT_H
#define _LCD_INIT_H


#include "pico/stdio.h"
#include "hardware/i2c.h"

#define I2C_PORT i2c0
```

```
#define I2C_SDA 4
#define I2C_SCL 5
#define LCD_ADDR 0x27 // Default Slave Address of the PCF8574T

void lcd_init(void);
void lcd_clear(void);
void lcd_set_cursor(uint8_t row, uint8_t col);
void lcd_print(const char *str);
void lcd_command(uint8_t cmd);
void lcd_data(uint8_t data);void lcd_init(void);

#endif /* _LCD_INIT_H */
```

Components.h

```
#ifndef _COMPONENTS_H
#define _COMPONENTS_H

#include <pico/stdlib.h>

#define DEBOUNCE_TIME 100

#define SWITCH1_PIN 0
#define SWITCH2_PIN 1
#define SWITCH3_PIN 2
#define SWITCH4_PIN 3
#define LOG_SWITCH  6

#define LED1_PIN    9
#define LED2_PIN    10
#define LED3_PIN    11
#define LED4_PIN    12
#define LED5_PIN    13

#define BUTTON_PIN  15

void set_switch(uint8_t switch_pin);
void initialize_leds(uint8_t led_pin);
void toggle_leds(uint8_t led_pin);

#endif /*_COMPONENTS_H*/
```

Chime.h

```
#ifndef _CHIME_H
#define _CHIME_H

#include <pico/stdio.h>
#include "hardware/spi.h"
#define SAMPLE_COUNT 64
#define SPI_PORT spi0
#define SPI_CS     17
#define SPI_SCK    18
#define SPI_MOSI   19

// sample_rate_hz = desired_frequency * SAMPLE_COUNT
// e.g. 440 Hz tone -> 440 * 256 = 112640
#define CHIME_SAMPLE_RATE (440 * SAMPLE_COUNT)

extern uint8_t sine_table[SAMPLE_COUNT * 2];

void generate_sine(void);
void spi_start(void);
void chime_start(int sample_rate_hz);
void chime_stop(void);

#endif /*_CHIME_H*/
```

ADC_Init.h

```
#ifndef _ADC_INIT_H
#define _ADC_INIT_H

#include "hardware/adc.h"

#define POT_PIN 27 // for ADC1
#define POT_ADC 1
#define PHO_PIN 26 // for ADC0
#define PHO_ADC 0
#define ADC_CONV_FACTOR (3.3f/((1 << 12) - 1))

// initialize the adc
void adc_start(void);

#endif /*_ADC_INIT_H*/
```

Main_Init.h

```
#ifndef _MAIN_INIT_H
#define _MAIN_INIT_H

#include "adc_init.h"
#include "components.h"

void main_init(void);

#endif /*_MAIN_INIT_H*/
```

Source Files:

Tasks.c

```
#include <stdio.h>
#include <stdlib.h>
#include "pico/stdlib.h"
#include "FreeRTOSConfig.h"
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"
#include "../main_init.h"
#include "../semaphores.h"
#include "../lcd_init.h"
#include "math.h"
#include "../chime.h"
#include "../tasks.h"

TaskHandle_t xProgressHandle  = NULL;
TaskHandle_t xButtonHandle    = NULL;
TaskHandle_t xSwitchHandle    = NULL;

// ---------------------- State Flags ---------------------- //
volatile bool start_flag      = 0;  // when program starts
volatile bool progress_flag   = 0;  // for the progress bar
volatile bool genre_flag      = 0;  // for genre chooing
volatile bool book_rec_flag   = 0;  // dark mode: book recommendation
volatile int  log_inx         = 0;  // index to track current location in datamap
volatile bool log_mode        = false; // suppress chime while browsing logs

// ---------------------- Photo State ---------------------- //
```

```
typedef enum { PHOTO_NORMAL, PHOTO_DARK, PHOTO_BRIGHT } photo_mode_t;
volatile photo_mode_t photo_mode    = PHOTO_NORMAL;
volatile bool         photo_enabled = true;

// ---------------------- Log into MAP ---------------------- //
char current_genre[16] = "Mystery";
int current_progress   = 0;
LibraryLog DataMap[MAX_MAP_SIZE];

// ---------------------- LUTs ---------------------- //

// Shared genre names — used by normal mode and dark (book rec) mode
static const char *genres[16] = {
   "Mystery",     "Romance",      "Horror",     "Fantasy",
   "Psychological","Sci-Fi",       "Classics",   "Historical",
   "Comedy",      "Philosophy",    "Classics",    "Biography",
   "Adventure",    "Manga/Comics",  "True Crime",  "LGBTQ+",
};

static const char *book_recs[16] = {
   "Gone Girl",       // Mystery
   "Pride&Prejudice",  // Romance
   "Pet Sematary",     // Horror
   "Mistborn",         // Fantasy
   "Fight Club",       // Psychological
   "Dune",             // Sci-Fi
   "1984",             // Classics
   "All Quiet...",     // Historical
   "Good Omens",       // Comedy
   "Fear&Trembling",   // Philosophy
   "Brave New World",  // Classics
   "Night",            // Biography
   "Into the Wild",    // Adventure
   "Frieren",          // Manga/Comics
   "In Cold Blood",    // True Crime
   "Heated Rivalry",  // LGBTQ+
};

// 15 new genres for bright mode
static const char *bright_genres[16] = {
   "Satire",
   "Thriller",
   "Self-Help",
   "Poetry",
   "Drama",
```

```
    "Western",
    "Dystopian",
    "Mythology",
    "Military",
    "Political",
    "Travel",
    "Art & Design",
    "Music",
    "Sports",
    "Cooking",
    "Nature",
};

// ---------------------- Tasks ---------------------- //
void Progress_Task(void *pvParameters) {
    bool rst = 0;
    int prev_leds_on = 0;
    char buf[16];
    for(;;) {
        if(progress_flag){
            xSemaphoreTake(xADCMutex, portMAX_DELAY);
            adc_select_input(POT_ADC);
            uint16_t raw = adc_read();
            xSemaphoreGive(xADCMutex);

            printf("Raw Value: 0x%03x, voltage = %f V\n", raw, raw * ADC_CONV_FACTOR);
            float voltage_adc = raw * ADC_CONV_FACTOR;
            float threshold = voltage_adc / 3.3f;
            int leds_on = (int)roundf(threshold * 5);

            xSemaphoreTake(xLedMutex, portMAX_DELAY);
            for(int i = LED1_PIN; i < LED1_PIN + 5; i++){
                if((i - LED1_PIN) < leds_on){ gpio_put(i, 1); }
                else{ gpio_put(i, 0); }
            }
            xSemaphoreGive(xLedMutex);

            if(abs(prev_leds_on - leds_on) >= 1){
                int percent = leds_on * 20;
                snprintf(buf, sizeof(buf), "Progress: %d%%", percent);
                current_progress = percent;
                lcd_clear();
                lcd_print(buf);
            }
            prev_leds_on = leds_on;
```

```
            rst = 1;
        }
        else{
            if(rst){
                lcd_clear();
                lcd_print("Registration");
                lcd_set_cursor(1, 0);
                lcd_print("Complete!");
                rst = 0;
            }
        }
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

void button_isr(uint gpio, uint32_t events){
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    xSemaphoreGiveFromISR(xButtonSemaphore, &xHigherPriorityTaskWoken);
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}

void Button_Task(void *pvParameters){
    for(;;){
        if(xSemaphoreTake(xButtonSemaphore, portMAX_DELAY) == pdTRUE){
            vTaskDelay(pdMS_TO_TICKS(100)); // debounce
            if(!gpio_get(BUTTON_PIN) && !log_mode){
                if(!start_flag){
                    // --- Entry: choose path based on light condition ---
                    start_flag = 1;
                    photo_enabled = false;
                    if(photo_mode == PHOTO_DARK){
                        book_rec_flag = 1;
                    }
                    // PHOTO_BRIGHT and PHOTO_NORMAL both use the normal progress/genre flow
                    // photo_mode stays frozen (photo_enabled=false) so Switch_Task can check it
                }
                else{
                    // --- Exit / advance: depends on active state ---
                    if(book_rec_flag){
                        // one button press exits book rec state
                        book_rec_flag = 0;
                        start_flag    = 0;
                        photo_enabled = true;
                    }
                    else{
```

```
                // Normal mode state machine: genre → progress → done
                if(!genre_flag && !progress_flag){
                  genre_flag = 1;
                }
                else if(genre_flag && !progress_flag){
                  genre_flag    = 0;
                  progress_flag = 1;
                }
                else{
                  progress_flag = 0;
                  start_flag    = 0;
                  photo_enabled = true;
                  DataMap[log_inx].progress = current_progress;
                  snprintf(DataMap[log_inx].genre, 16, "%s", current_genre);
                  log_inx = (log_inx + 1) % MAX_MAP_SIZE;
                  xSemaphoreGive(xChimeSemaphore);
                }
              }
            }
          }

          xSemaphoreTake(xLedMutex, portMAX_DELAY);
          for(int i = LED1_PIN; i < LED1_PIN + 5; i++) gpio_put(i, 0);
          xSemaphoreGive(xLedMutex);

          while(xSemaphoreTake(xButtonSemaphore, 0) == pdTRUE);
        }
      }
    }

    void Switch_Task(void *pvParameters){
      uint8_t last_state = 0xFF; // used to only refresh LCD when necessary
      uint8_t log_switch;

      // different genres
      typedef enum { SW_BOOK_REC, SW_GENRE, SW_IDLE } sw_mode_t;
      sw_mode_t prev_mode = SW_IDLE;
      bool use_bright = false;
      const char* genre;

      for(;;){
        uint8_t switches = 0;
        switches |= gpio_get(SWITCH1_PIN) << 0;
        switches |= gpio_get(SWITCH2_PIN) << 1;
        switches |= gpio_get(SWITCH3_PIN) << 2;
```

```
    switches |= gpio_get(SWITCH4_PIN) << 3;

    // Determine current mode
    sw_mode_t cur_mode;
    if(book_rec_flag) cur_mode = SW_BOOK_REC;
    else if(genre_flag) cur_mode = SW_GENRE;
    else                cur_mode = SW_IDLE;

    // Reset last_state on mode transition so display refreshes immediately
    if(cur_mode != prev_mode){
        last_state = 0xFF;
        if(prev_mode == SW_BOOK_REC){
            lcd_clear();
            lcd_print("Luminous Library");
        }
        if(cur_mode == SW_GENRE){
            // Snapshot photo_mode once so bright vs normal stays stable
            use_bright = (photo_mode == PHOTO_BRIGHT);
        }
        prev_mode = cur_mode;
    }

    if(cur_mode == SW_BOOK_REC){
        if(switches != last_state){
            last_state = switches;
            lcd_clear();
            lcd_print(genres[switches]);
            lcd_set_cursor(1, 0);
            lcd_print(book_recs[switches]);
        }
    }
    else if(cur_mode == SW_GENRE){
        if(switches != last_state){
            last_state = switches;
            lcd_clear();
            genre = use_bright ? bright_genres[switches] : genres[switches];
            lcd_print(genre);
            snprintf(current_genre, 16, "%s", genre);
        }
    }
    else{ // SW_IDLE
    }

    vTaskDelay(pdMS_TO_TICKS(50));
}
```

```
}

void Chime_Task(void *pvParameters){
  for(;;){
    if(xSemaphoreTake(xChimeSemaphore, portMAX_DELAY) == pdTRUE){
      if(!log_mode){
        chime_start(CHIME_SAMPLE_RATE);
        while(!start_flag && !log_mode){
          vTaskDelay(pdMS_TO_TICKS(100));
        }
        chime_stop();
      }
    }
  }
}

void Photo_Task(void *pvParameters){
  for(;;){
    if(photo_enabled){
      xSemaphoreTake(xADCMutex, portMAX_DELAY);
      adc_select_input(PHO_ADC);
      uint16_t raw = adc_read();
      xSemaphoreGive(xADCMutex);

      float voltage = raw * ADC_CONV_FACTOR;
      printf("Photo: 0x%03x, %.3f V\n", raw, voltage);

      if(voltage >= 2.0f){
        photo_mode = PHOTO_DARK;
      } else if(voltage < 0.5f){
        photo_mode = PHOTO_BRIGHT;
      } else {
        photo_mode = PHOTO_NORMAL;
      }
    }
    vTaskDelay(pdMS_TO_TICKS(200));
  }
}


void Log_Task(void *pvParameters) {
  for (;;) {
    // Wait until GP6 is high
    if (gpio_get(LOG_SWITCH)) {
```

```
            vTaskDelay(pdMS_TO_TICKS(50)); // Debounce
            if (gpio_get(LOG_SWITCH)) {

                log_mode = true;
                chime_stop();

                if (log_inx == 0) {
                    lcd_clear();
                    lcd_print("No Logs Found");
                    vTaskDelay(pdMS_TO_TICKS(1500));
                } else {
                    int i = 0;
                    while (i < log_inx && gpio_get(LOG_SWITCH)) {
                        lcd_clear();
                        lcd_print(DataMap[i].genre);

                        lcd_set_cursor(1, 0);
                        char buf[16];
                        snprintf(buf, sizeof(buf), "Prog: %d%%", DataMap[i].progress);
                        lcd_print(buf);

                        i++;
                        vTaskDelay(pdMS_TO_TICKS(1500));
                    }
                }

                // Wait for switch release before restoring display
                while(gpio_get(LOG_SWITCH)){
                    vTaskDelay(pdMS_TO_TICKS(50));
                }
                log_mode = false;
                lcd_clear();
                lcd_print("Luminous Library");

            }
        }
        // Check every 100ms so we don't waste CPU
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

Semaphores.c

```
#include "FreeRTOS.h"
```

```c
#include "pico/stdlib.h"
#include "semphr.h"

SemaphoreHandle_t xButtonSemaphore;
SemaphoreHandle_t xLedMutex;
SemaphoreHandle_t xChimeSemaphore;
SemaphoreHandle_t xADCMutex;
```

LCD_Init.c

```c
#include "pico/stdlib.h"
#include "../lcd_init.h"

// got this from claude, but I know how it works

static void lcd_write(uint8_t data){
   i2c_write_blocking(I2C_PORT, LCD_ADDR, &data, 1, false);
}

static void lcd_pulse_enable(uint8_t data){
   lcd_write(data | 0x04);  // EN high
   sleep_us(1);
   lcd_write(data & ~0x04); // EN low
   sleep_us(50);
}

static void lcd_send_nibble(uint8_t nibble, uint8_t mode){
   uint8_t data = (nibble & 0xF0) | mode | 0x08; // 0x08 = backlight on
   lcd_pulse_enable(data);
}

static void lcd_send_byte(uint8_t data, uint9_t mode){
   lcd_send_nibble(data & 0xF0, mode);       // upper nibble
   lcd_send_nibble((data << 4) & 0xF0, mode); // lower nibble
}

void lcd_command(uint8_t cmd){
   lcd_send_byte(cmd, 0x00); // RS=0
}

void lcd_data(uint8_t data){
   lcd_send_byte(data, 0x01); // RS=1
}
```

```c
void lcd_init(void){
    i2c_init(I2C_PORT, 100 * 1000);
    gpio_set_function(I2C_SDA, GPIO_FUNC_I2C);
    gpio_set_function(I2C_SCL, GPIO_FUNC_I2C);
    gpio_pull_up(I2C_SDA);
    gpio_pull_up(I2C_SCL);

    sleep_ms(50);

    // force 8-bit mode 3 times
    lcd_send_nibble(0x30, 0x00);
    sleep_ms(5);
    lcd_send_nibble(0x30, 0x00);
    sleep_us(150);
    lcd_send_nibble(0x30, 0x00);

    // switch to 4-bit mode
    lcd_send_nibble(0x20, 0x00);

    // function set: 4-bit, 2 lines, 5x8 font
    lcd_command(0x28);
    // display on, cursor off, blink off
    lcd_command(0x0C);
    // clear display
    lcd_command(0x01);
    sleep_ms(2);
    // entry mode: increment, no shift
    lcd_command(0x06);
}

void lcd_clear(void){
    lcd_command(0x01);
    sleep_ms(2);
}

void lcd_set_cursor(uint8_t row, uint8_t col){
    uint8_t offsets[] = {0x00, 0x40};
    lcd_command(0x80 | (offsets[row] + col));
}

void lcd_print(const char *str){
    while(*str){
        lcd_data(*str++);
    }
}
```

Components.c

```c
#include "pico/stdlib.h"
#include "../components.h"

void set_switch(uint8_t switch_pin){
   // default direction is input
   gpio_init(switch_pin);

   // set pin to have internal pullup - unnecessary
   gpio_pull_up(switch_pin);
}

void initialize_leds(uint8_t led_pin){
   gpio_init(led_pin);
   gpio_set_dir(led_pin, GPIO_OUT);
}

void toggle_leds(uint8_t led_pin){
   gpio_xor_mask64(1u << led_pin);
}
```

Chime.c

```c
#include <pico/stdio.h>
#include "../chime.h"
#include "hardware/gpio.h"
#include <pico/time.h>
#include "math.h"

#ifndef M_PI
#define M_PI 3.14159265358979323846f
#endif

uint8_t sine_table[SAMPLE_COUNT * 2];

static volatile int sample_index = 0;
static struct repeating_timer dac_timer;

static bool dac_sample_irq(struct repeating_timer *t){
   gpio_put(SPI_CS, 0);
   spi_write_blocking(SPI_PORT, &sine_table[sample_index * 2], 2);
```

```
    gpio_put(SPI_CS, 1);
    sample_index = (sample_index + 1) % SAMPLE_COUNT;
    return true;
}

void chime_start(int sample_rate_hz){
    sample_index = 0;
    int period_us = 1000000 / sample_rate_hz;
    add_repeating_timer_us(-period_us, dac_sample_irq, NULL, &dac_timer);
}

void chime_stop(void){
    cancel_repeating_timer(&dac_timer);
    gpio_put(SPI_CS, 1);
}

void spi_start(void){
    spi_init(SPI_PORT, 2000 * 1000);
    spi_set_format(SPI_PORT, 8, SPI_CPOL_0, SPI_CPHA_0, SPI_MSB_FIRST);
    // set all the used pins
    gpio_set_function(SPI_SCK,  GPIO_FUNC_SPI);
    gpio_set_function(SPI_MOSI, GPIO_FUNC_SPI);
    gpio_init(SPI_CS);

    // set the CS to be an output
    gpio_set_dir(SPI_CS, GPIO_OUT);

    // CS idles as high
    gpio_put(SPI_CS, 1);
}

void generate_sine(void){
    for(int i = 0; i < SAMPLE_COUNT; i++){
        float s = sinf(2.0f * M_PI * i / SAMPLE_COUNT);
        uint16_t sample = (uint16_t)((s + 1.0f) / 2.0f * 1023.0f); // 10-bit
        // LTC1661: [A3 A2 A1 A0 | D9..D0 | X X]
        // 0xF000 = write+update both DACs, data in bits 11-2
        uint16_t word = 0xF000 | ((sample & 0x3FF) << 2);
        sine_table[i * 2]     = (word >> 8) & 0xFF;
        sine_table[i * 2 + 1] = word & 0xFF;
    }
}
```

ADC_Init.c

```c
#include "hardware/adc.h"
#include "../adc_init.h"

void adc_start(void){
   adc_init();
   adc_gpio_init(POT_PIN);
   adc_gpio_init(PHO_PIN);
}
```

Main_Init.c

```c
#include "../components.h"
#include "../adc_init.h"
#include "../main_init.h"
#include "../lcd_init.h"
#include "../chime.h"

void main_init(void){
   for(int i = LED1_PIN; i < LED1_PIN + 5; i++){initialize_leds(i);}
   for(int i = SWITCH1_PIN; i < SWITCH1_PIN + 4; i++){set_switch(i);}
   set_switch(LOG_SWITCH);
   set_switch(BUTTON_PIN);
   adc_start();
   lcd_init();
   spi_start();
   generate_sine();
}
```